Sri Lanka Institute of Information Technology



IT3101 – Data Warehousing and Business Intelligence

Assignment Report

Olist Brazilian E-Commerce

Data Warehouse and Business Intelligence Solution

Data Warehouse Design · ETL Development · OLAP Cube · Power BI Reporting

Group ID: 2026-DS-02

Lab Group: Y3.S1. WD. DS. 0102

Group Members:

Student ID	Name
IT24104110	Thulmanthi W.A.S.
IT24103860	Karunathilaka S.H.T.R.
IT24103869	Thennakoon T.M.S.D.
IT24104178	Jayasuriya J.A.Y.N.

Introduction

Data warehousing is a core part of modern business intelligence. It lets an organization bring large volumes of historical operational data into one consistent analytical repository that is designed for querying rather than for processing transactions. By integrating data from several sources, cleansing it, and storing it in a dimensional model, a data warehouse helps an organization identify trends, monitor performance, and produce the reports that support strategic and operational decisions.

This project designed and implemented a complete data warehousing and business intelligence solution on the Microsoft BI stack. Microsoft SQL Server was used as the database platform, SQL Server Integration Services (SSIS) and T-SQL for the ETL process, SQL Server Analysis Services (SSAS) as the OLAP engine, and Power BI Desktop for reporting and visualization.

The dataset is the Olist Brazilian E-Commerce Public Dataset, which contains about 100,000 orders placed on the Olist marketplace in Brazil between September 2016 and October 2018. It is distributed as nine related CSV files covering orders, order items, customers, sellers, products, payments, reviews, geolocation and product category translations. Because these files are normalised and linked through business keys, they represent a realistic operational (OLTP) system. The seven transactional tables were imported into a SQL Server database to act as the operational source. The solution also uses two other source types: CSV reference files for geolocation and category translation, and a JSON feed of Brazilian public holidays retrieved from the Nager.Date web API. Profiling the data showed issues such as one customer appearing under many order-level identifiers, orders with no delivery date, products without a category, and a state-only holiday inside the national holiday list. Rules for each were defined and applied during transformation.

The dimensional model contains two fact tables, Fact_Sales (one row per order item) and Fact_OrderFulfilment (one row per order). They share conformed dimensions: Date, Customer, Product, Seller, Location, OrderStatus and PaymentType. Two fact tables were used because delivery, payment and review values belong to the whole order, and storing them on every item row would distort averages. Because the fact tables share dimensions, the model is a fact constellation (galaxy schema). The Location dimension is a role-playing dimension, used once for the customer location and once for the seller location, so that where an order is shipped from can be compared with where it is delivered to.

The warehouse is built in a single SQL Server database organised into four schemas: stg for staging, dw for the warehouse, mart for data marts, and etl for logging. A master SSIS package runs three packages in order: extraction of all sources into staging, loading of the dimensions, and loading of the fact tables. The ETL uses a repeatable full load, in which each run clears and rebuilds the data. The holiday data is retrieved by an SSIS Script Task, with a saved copy of the raw JSON used as a fallback if the API cannot be reached. A validation procedure runs sixteen automated checks after every load, comparing row counts and revenue, freight and payment totals against the source. All checks passed on repeated runs. Two department data marts were then created: a Sales Mart for sales and marketing, and a Delivery and Satisfaction Mart for operations. On top of the warehouse, an SSAS multidimensional cube was built with user hierarchies and two measure groups, and a three-page Power BI report was developed with an Executive Summary, a Trend Analysis and an Interactive Analysis page.

This report documents the full solution. It covers the dataset and data sources, the architecture, the dimensional model, ETL development and validation, the data marts, the OLAP and dashboard layer, and the business insights and recommendations drawn from the results.

Part A – Data Warehouse Design and ETL Development

Task 1 – Dataset Selection

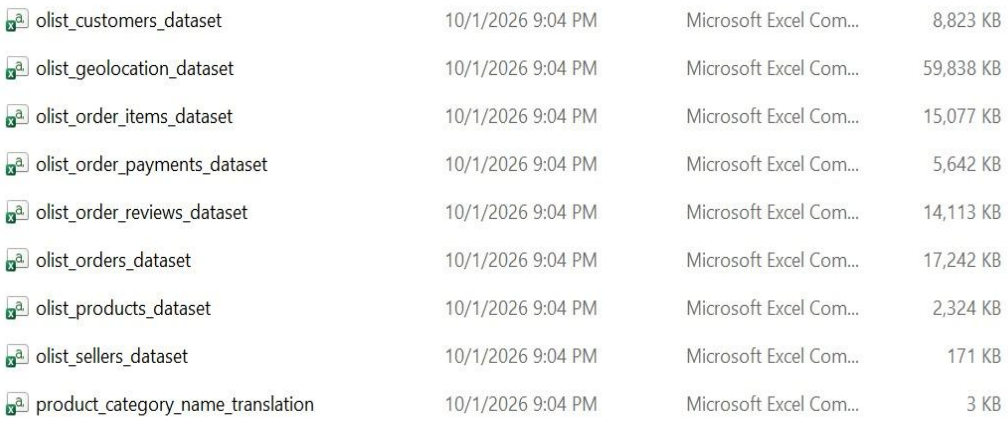
1.1 Selected Dataset

Dataset: Olist Brazilian E-Commerce Public Dataset
Source: Kaggle (Olist Store, Brazil)

The selected dataset contains detailed transactional information about orders placed on the Olist marketplace, a Brazilian e-commerce platform that connects small and medium sellers to the major online marketplaces in Brazil. The data covers roughly **100,000** orders that represents genuine operational data captured by the marketplace order management system.

Item	Description
Business domain	Retail / E-commerce. An online marketplace in Brazil.
System represented	The order management system of Olist. It records customers, orders, items, payments, delivery and reviews. This is an OLTP (online transaction processing) system.
Dataset name	Brazilian E-Commerce Public Dataset by Olist
Dataset source	Published by Olist on Kaggle: https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce
Time period	Orders placed from 4 September 2016 to 17 October 2018
Dataset format	9 CSV files (comma-separated text files)
Size	About 100,000 orders. About 1.55 million rows across the 9 files.
Currency	Brazilian real (BRL)
Privacy	The data is anonymised. Customer and seller identifiers are encoded, and company names in review text are replaced.
Licence	CC BY-NC-SA 4.0 (non-commercial use, suitable for coursework)

- 9 CSV files



1.2 Dataset Attributes

The following table lists the nine source files and the key attributes that each contributes to the data warehouse.

Source File	Description	Key Attributes
olist_orders_dataset.csv	Order header records, one row per order	order_id, customer_id, order_status, order_purchase_timestamp, order_approved_at, order_delivered_carrier_date, order_delivered_customer_date, order_estimated_delivery_date
olist_order_items_dataset.csv	Order line items, one row per item on an order	order_id, order_item_id, product_id, seller_id, shipping_limit_date, price, freight_value
olist_customers_dataset.csv	Customer master data	customer_id, customer_unique_id, customer_zip_code_prefix, customer_city, customer_state
olist_sellers_dataset.csv	Seller master data	seller_id, seller_zip_code_prefix, seller_city, seller_state
olist_products_dataset.csv	Product master data and physical attributes	product_id, product_category_name, product_name_lenght, product_description_lenght, product_photos_qty, product_weight_g, product_length_cm, product_height_cm, product_width_cm
olist_order_payments_dataset.csv	Payment transactions for each order	order_id, payment_sequential, payment_type, payment_installments, payment_value
olist_order_reviews_dataset.csv	Customer satisfaction reviews	review_id, order_id, review_score, review_comment_title, review_comment_message, review_creation_date, review_answer_timestamp
olist_geolocation_dataset.csv	Brazilian postcode to latitude and longitude reference data	geolocation_zip_code_prefix, geolocation_lat, geolocation_lng, geolocation_city, geolocation_state
product_category_name_translation.csv	Portuguese to English product category lookup	product_category_name, product_category_name_english

1.3 Justification for Selection

The Olist Brazilian E-Commerce dataset was selected because it provides a realistic OLTP source with enough records and attributes for dimensional modelling, and it is publicly available and licensed for academic use. The main reasons are set out below.

1. A genuine OLTP source with sufficient volume

The data comes from the order management system of a real online marketplace and is stored in nine normalised tables linked by business keys such as order_id, product_id and seller_id. It holds 99,441 orders, 112,650 order items, 103,886 payments and 99,224 reviews, enough for meaningful trends yet small enough to reload repeatedly during ETL development. The seven transactional tables were imported into SQL Server as the operational source.

2. Multi-source integration

The solution uses three source types: the SQL Server operational database, CSV reference files for geolocation and category translation, and a JSON feed of Brazilian public holidays from the Nager.Date web API. Combining them requires joining tables, matching zip codes, categories and dates across sources, and resolving attributes to surrogate keys.

3.Attributes that support a fact constellation

The data separates clearly into measures (price, freight value, payment value, instalments, review score and delivery duration) and dimensions (date, customer, product, seller, location, order status and payment type). Sales are recorded per order item, while delivery, payment and review values belong to the whole order. This gives two fact tables at different grains, Fact_Sales and Fact_OrderFulfilment, sharing conformed dimensions.

4. Time and geographic analysis

Order timestamps (purchase, approval, carrier handover, delivery and estimated delivery) support a Year, Quarter, Month and Day hierarchy, delivery delay measures and holiday analysis. Customers and sellers both carry zip code, city and state, so one Location dimension can play two roles, customer location and seller location.

5. Realistic business questions

The dataset answers questions managers would actually ask:

- How do orders and revenue change over time?
- Which categories, sellers and states generate the most revenue?
- How long does delivery take, and how often are orders late?
- Do late deliveries lead to lower review scores?

6. Data quality challenges that justify an ETL process

customer_id changes with every order, so customer_unique_id identifies repeat buyers. Some orders lack delivery dates, some products lack a category, and the holiday feed includes a state-only holiday that must be filtered out.

Limitations

There is no cost or profit data, so profit margin cannot be measured. The data covers about two years, and the first and last months hold very few orders.

Task 2 – Data Source Identification and Preparation

This task identifies the data sources used in the solution, describes their format and attributes, explains how the sources relate to each other, documents the preparation work that was carried out, and explains how the data from the different sources is brought together in one analytical environment.

2.1 Overview of the Data Sources

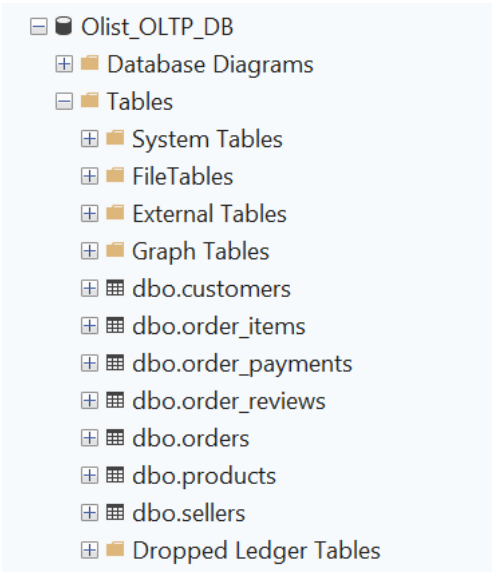
The solution uses three sources of three different types: a relational database, CSV files and a JSON web API. Using several source types reflects a real organization, where analytical data rarely comes from a single place.

Source	Type of format	Records	Used for
S1: Olist operational database (Olist_OLTP_DB)	Relational database, Microsoft SQL Server	7 tables, 550,688 rows	Business transactions: customers, orders, order items, payments, reviews, products and sellers
S2: Olist reference files	CSV flat files, UTF-8	Geolocation 1,000,163 rows. Category translation 71 rows.	Latitude and longitude of locations, and English product category names
S3: Nager.Date public holiday service	JSON, retrieved from a REST API over HTTPS	42 holidays (Brazil, 2016 to 2018)	Holiday information for the Date dimension

2.2 Source S1: Olist Operational Database

Olist is a Brazilian e-commerce platform that connects small sellers with customers through an online marketplace. Its order management system records each customer order together with the products, sellers, payments, delivery dates and customer reviews. The data was anonymised before publication: customer and seller identifiers are encoded, and company names in review text are replaced. The orders in the dataset were placed between September 2016 and October 2018.

In this project the seven transactional tables are stored in the SQL Server database Olist_OLTP_DB, which acts as the operational source.



Tables of the Olist_OLTP_DB database

Table	Rows	Primary key	Main attributes
customers	99,441	customer_id	customer_id, customer_unique_id, customer_zip_code_prefix, customer_city, customer_state
sellers	3,095	seller_id	seller_id, seller_zip_code_prefix, seller_city, seller_state
products	32,951	product_id	product_id, product_category_name, product_name_lenght, product_description_lenght, product_photos_qty, product_weight_g, product_length_cm, product_height_cm, product_width_cm
orders	99,441	order_id	order_id, customer_id, order_status, order_purchase_timestamp, order_approved_at, order_delivered_carrier_date, order_delivered_customer_date, order_estimated_delivery_date
order_items	112,650	order_id and order_item_id	order_id, order_item_id, product_id, seller_id, shipping_limit_date, price, freight_value
order_payments	103,886	order_id and payment_sequential	order_id, payment_sequential, payment_type, payment_installments, payment_value
order_reviews	99,224	None (review_id repeats)	review_id, order_id, review_score, review_comment_title, review_comment_message, review_creation_date, review_answer_timestamp

The table definitions use data types that suit the data. Identifiers are CHAR(32) because they are fixed 32-character codes. Names use NVARCHAR so that Portuguese accents are stored correctly. Timestamps use DATETIME2(0) and money uses DECIMAL(10,2). Zip code prefixes are stored as text, because many begin with zero.

Points about the data that affect the design:

- One order can contain several items, and can be paid for with several payments.
- **customer_id changes with every order.** The field customer_unique_id identifies the real person. The customers table has 99,441 rows but only 96,096 distinct people.
- All monetary values are in Brazilian reais (BRL).
- Two source columns are misspelled (product_name_lenght and product_description_lenght). They are kept as they are in the source database and corrected in the warehouse.

2.3 Source S2: Reference Files (CSV)

Two CSV files supply reference data that the transactional tables do not contain. The geolocation file lists latitude and longitude points for Brazilian zip code prefixes. The category translation file gives the English name for each Portuguese product category.

Reference files

File	Rows	Attributes	Purpose
olist_geolocation_dataset.csv	1,000,163	geolocation_zip_code_prefix, geolocation_lat, geolocation_lng, geolocation_city, geolocation_state	Coordinates for the Location dimension
product_category_name_translation.csv	71	product_category_name, product_category_name_english	English category names for the Product dimension

2.4 Source S3: Public Holiday API (JSON)

Nager.Date is a free public holiday service. It needs no registration or key. One request returns the public holidays of one country and one year as a JSON array. The request used for Brazil has this form, where the year is changed from 2016 to 2018:

```
https://date.nager.at/api/v3/PublicHolidays/2017/BR
```

Each holiday in the response is one JSON object. The first record returned for 2016 is shown below.


```
{"date":"2016-01-01","localName":"Confraternização Universal",
 "name":"New Year's Day","countryCode":"BR","fixed":false,
 "global":true,"counties":null,"launchYear":null,"types":["Public"]}
```


Fields of the holiday API response

Field	Data type	Meaning
date	Date	The date of the holiday
localName	Text	The holiday name in Portuguese
name	Text	The holiday name in English
countryCode	Text	Country code, BR for Brazil
fixed	True or false	Whether the holiday falls on the same date every year
global	True or false	True if the holiday applies to the whole country, false if only some states observe it
counties	List	The states where the holiday applies, when it is not nationwide (can be empty)
launchYear	Number	The year the holiday began (can be empty)
types	List	The kind of holiday, for example Public

2.5 Relationships Between the Sources

The tables inside S1 are linked by order, customer, product and seller identifiers. The two outside sources connect to S1 through three shared attributes: the zip code prefix, the product category name and the calendar date

Relationships between tables and sources

From	To	Join attribute	Relationship
orders (S1)	customers (S1)	customer_id	Many to one
order_items (S1)	orders (S1)	order_id	Many to one
order_items (S1)	products (S1)	product_id	Many to one
order_items (S1)	sellers (S1)	seller_id	Many to one
order_payments (S1)	orders (S1)	order_id	Many to one
order_reviews (S1)	orders (S1)	order_id	Many to one, usually one review per order
customers, sellers (S1)	geolocation (S2)	Zip code prefix	Many to one, after geolocation is reduced to one row per prefix
products (S1)	category translation (S2)	product_category_name	Many to one
orders (S1)	holidays (S3)	Purchase date equals holiday date	Many orders to one holiday date, through the Date dimension

2.6 Data Preparation Steps

1. **Project folders.** A project folder was created with sub-folders for the source CSV files, the holiday JSON files, SQL scripts, SSIS projects and report material.
2. **Download and inspection.** The nine CSV files were downloaded from Kaggle and extracted into the source folder. All nine files were present. They were not opened in Excel, because Excel can change dates and codes. A text editor was used instead.
3. **Creating the operational database.** The database Olist_OLTP_DB was created in SQL Server, with one table for each transactional file. Primary keys and foreign keys were defined so that the database behaves like a real OLTP system. The reviews table has no primary key because review_id is not unique.
4. **Importing the data.** The seven CSV files were imported with the T-SQL command BULK INSERT, using the options FORMAT = CSV, a double quote as field quote, FIRSTROW = 2 to skip the header, code page 65001 (UTF-8) so that Portuguese accents load correctly, and KEEPNULLS so that empty values stay empty. Parent tables were loaded before child tables so that the foreign keys were satisfied. This step builds the source system. It is not part of the ETL process.

The reviews file failed on the first attempt with an OLE DB provider error. The cause was the fixed line-ending option. After this option was removed, so that SQL Server detected the line endings itself, all 99,224 rows loaded.

5. **Checking the load.** Row counts were compared with the published counts of the dataset .They match, so no rows were lost.
6. **Reference files.** The geolocation and category translation files were kept as CSV files in the source folder. SSIS reads them as UTF-8 text with the header row skipped.
7. **Holiday source.** The API request was tested in a web browser. The responses for 2016, 2017 and 2018 were saved as raw JSON files in a separate folder and checked in a text editor, which confirmed UTF-8 encoding. The SSIS package later calls the API directly and refreshes these files, and uses the saved files if the service cannot be reached (Task 5).
8. **Profiling.** The data was profiled with SQL queries to find quality problems before any transformation was designed (Section 2.7).
9. **Staging.** A staging area was created in the warehouse database for all three sources. After the first SSIS extraction, the staging tables contained the same number of rows as the sources (Section 2.8).

2.7 Data Profiling and Quality Findings

Data quality findings and planned handling

	Finding	Quality Dimension	Evidence	Planned handling
1	Partial months at the start and end of the data	Completeness	Orders per month: 2016-09 = 4, 2016-10 = 324, 2016-12 = 1, no orders in 2016-11, 2018-09 = 16, 2018-10 = 4.	Load all data. Analysis focuses on the full months, January 2017 to August 2018.
2	Orders that are not real sales	Validity (Business rule)	Statuses: delivered 96,478, shipped 1,107, canceled 625, unavailable 609, invoiced 314, processing 301, created 5, approved 2.	Keep the status. Mark canceled and unavailable orders as not counting as sales.
3	Missing dates in orders	Completeness	160 without approval date, 1,783 without carrier date, 2,965 without delivery date.	Keep the rows. Calculate delivery measures only when a delivery date exists.
4	Delivered orders without a delivery date	Completeness	8 orders. No order was delivered before it was purchased.	Keep the rows, leave delivery measures empty, and report them in validation.
5	Several customer identifiers per person	Uniqueness	99,441 customer rows but 96,096 distinct customer_unique_id values.	One Customer dimension row per customer_unique_id.

	Finding	Quality Dimension	Evidence	Planned handling
6	Reviews are not one per order	Uniqueness	99,224 rows, 98,410 distinct review_id values, 98,673 orders with a review.	Keep the latest review of each order. Orders without a review keep an empty score.
7	Missing product information	Completeness	610 products without a category, 2 without weight.	Category becomes Unknown. Weight stays empty.
8	Categories without an English name	Consistency	Two categories are missing from the translation file: pc_gamer and portateis_cozinha_e_preparadores_de_alimentos.	Add an English name for these two categories.
9	Unclear payment type	Validity	Payment types: credit_card 76,795, boleto 19,784, voucher 5,775, debit_card 1,529, not_defined 3.	Keep not_defined and group it as Not defined.
10	Prices and freight	Validity	Price 0.85 to 6,735.00. Freight 0.00 to 409.68. No negative values.	No change. Free shipping is valid.
11	Geolocation points outside Brazil	Accuracy	1,000,163 rows for 19,015 zip prefixes. 42 points lie outside Brazil. No missing coordinates, and all prefixes have 5 characters.	Remove the 42 points, then average the coordinates for each zip prefix.
12	Zip prefixes with no geolocation row	Consistency	157 customer zip prefixes and 7 seller zip prefixes.	Keep these locations with empty coordinates.
13	State-level holidays in the holiday list	Validity	3 of the 42 holidays are not nationwide.	Flag only nationwide holidays in the Date dimension.
14	Text encoding	Consistency	Portuguese accents display correctly in the reviews after import. Holiday names first appeared garbled when loaded with the wrong encoding.	Read all text files as UTF-8.

2.8 Integrating the Sources into One Analytical Environment

The three sources differ in type and format, and in a real company they would belong to different systems. They are combined into one analytical environment in four ways:

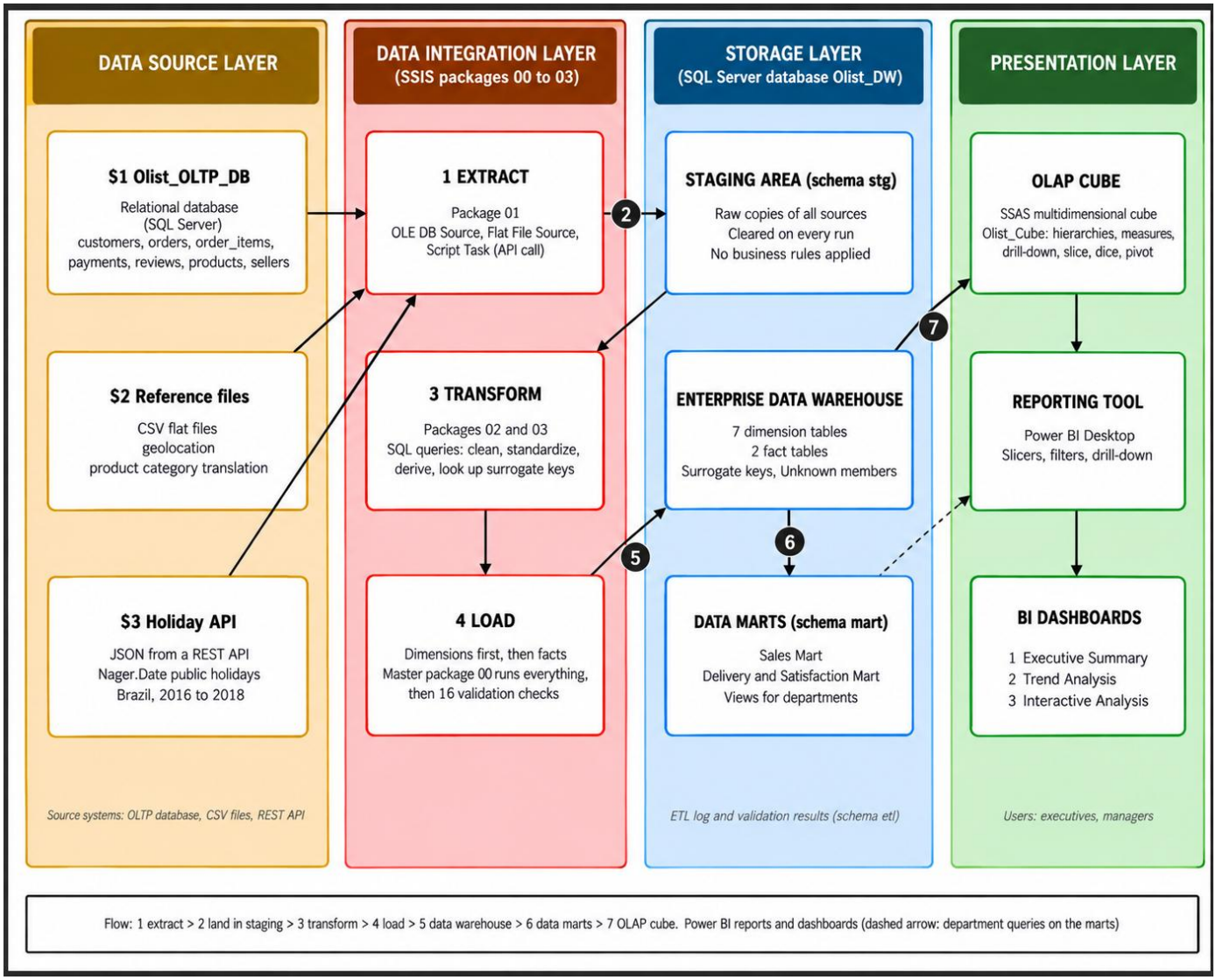
- **One landing area.** The database, CSV and JSON data are first copied into staging tables in the warehouse database, so the same SQL can handle every source.
- **Common keys and formats.** Zip code prefixes are kept as text of equal length so leading zeros are not lost, product categories are matched with the translation file, and dates are converted to a standard date key.

- **Shared dimensions.** Location merges customer and seller zip codes with geolocation coordinates, Product adds English category names, and Date is a full calendar enriched with holiday flags from the API.
- **Surrogate keys.** Each dimension has its own warehouse key, so the model does not depend on source system identifiers.

The result is one consistent warehouse in which sales, delivery, payment, review, location and holiday data can be analysed together.

Task 3: Data Warehouse and Business Intelligence Architecture

3.1 Architecture Diagram



Summary of the Architecture

Layers, components and technologies

Layer	Component	Technology	What was built
Data source	Operational database (S1)	SQL Server database Olist_OLTP_DB	7 transactional tables with primary and foreign keys
Data source	External files (S2)	CSV files	Geolocation and category translation files
Data source	External web service (S3)	JSON from the Nager.Date REST API	Brazilian public holidays for 2016 to 2018
Data source	Application systems	None	The case has no separate application system
Data integration	Extraction	SSIS: OLE DB Source, Flat File Source, Script Task	Package 01 copies all sources into staging
Data integration	Transformation	T-SQL source queries inside SSIS data flows	Cleaning, standardisation, derived attributes and key lookups
Data integration	Loading	SSIS packages 02, 03 and master package 00	Dimensions first, then facts, then 16 validation checks
Storage	Staging area	Schema stg in Olist_DW	Raw copies of all sources, cleared on every run
Storage	Enterprise data warehouse	Schema dw in Olist_DW	7 dimension tables and 2 fact tables
Storage	Data marts	Schema mart in Olist_DW	Sales Mart and Delivery and Satisfaction Mart
Storage	ETL control and log	Schema etl in Olist_DW	Run log, validation results and the validation procedure
Presentation	OLAP	SSAS multidimensional cube Olist_Cube	Hierarchies, measures and OLAP operations
Presentation	Reporting tool	Power BI Desktop	Interactive report with slicers and drill-down
Presentation	BI dashboards	Power BI report pages	Executive Summary, Trend Analysis and Interactive Analysis

3.2 Data Source Layer

The data source layer holds the systems and files that the warehouse is built from. Three sources of three different types are used (see Task 2 for the full description).

- **Operational database (S1).** The Olist order management data is stored in the SQL Server database Olist_OLTP_DB. It has seven related tables: customers, sellers, products, orders, order items, order payments and order reviews. The data was imported from the dataset files once, and the database has primary and foreign keys so that it behaves like a real transactional system. It holds 550,688 rows in total.
- **External files (S2).** The geolocation file (1,000,163 rows) and the product category translation file (71 rows) are kept as CSV files in UTF-8 encoding. They supply coordinates and English category names that the database does not contain.
- **External web service (S3).** The Nager.Date holiday service returns the public holidays of Brazil as JSON. The years 2016, 2017 and 2018 give 42 holidays. The raw responses are saved as JSON files and used as a backup.
- **Application systems.** No separate application system exists in this case, so none is used.

3.3 Data Integration Layer

SSIS packages

Package	Purpose	Main tasks
01_Staging_Load	Extraction	Clears the staging tables. Copies the 7 database tables with OLE DB data flows. Loads the 2 CSV files with Flat File data flows. A Script Task downloads the holiday JSON from the API. An Execute SQL Task reads the JSON into a staging table.
02_Load_Dimensions	Transformation and loading of dimensions	Clears the warehouse tables. Loads Dim_Date, Dim_OrderStatus, Dim_PaymentType, Dim_Seller, Dim_Customer, Dim_Product and Dim_Location from SQL queries over staging.
03_Load_Facts	Transformation and loading of facts	Clears the fact tables. Loads Fact_Sales and Fact_OrderFulfilment, looking up the dimension keys.
00_Master	Orchestration and validation	Runs packages 01, 02 and 03 in order, then runs the validation procedure.

Extraction. Seven OLE DB data flows read the database tables from Olist_OLTP_DB, and two Flat File data flows read the CSV files as UTF-8 text, skipping the header row. A C# Script Task requests the holiday data for each year and saves the raw JSON. If the service cannot be reached, it logs a warning and uses the saved file, so the package does not fail without an internet connection. All data is written to staging with no business rules.

Transformation. T-SQL queries serve as the sources of the SSIS data flows. They handle missing values, remove duplicates, standardise names and codes, convert data types, aggregate items and payments, and create derived attributes such as delivery days, the late-delivery flag, regions and the holiday flag.

Dimension keys are looked up in the same queries, and missing values map to the Unknown member (key -1).

Loading. Dimensions are loaded first so their surrogate keys exist, then the fact tables. Each run clears the warehouse tables first, which makes the process repeatable. The master package then runs a validation procedure with 16 checks and stores the results in the etl schema.

Load strategy. The ETL uses a full load because the dataset is a fixed historical snapshot (September 2016 to October 2018), so no new records arrive between runs. A full load gives the same result every time and avoids loading a period twice or missing one. A system receiving new orders daily would need an incremental load, noted as a possible future improvement.

3.4 Storage Layer

All storage-layer components live in one SQL Server database, Olist_DW. Four schemas separate the layers, so each component is easy to find and check.

- **Staging area (schema stg).** It has one table for each extracted source (customers, sellers, products, orders, order items, order payments, order reviews, geolocation, category translation and public holidays) plus a table that holds the raw holiday JSON. The tables hold raw copies, have no keys, and are cleared at the start of each run. Review comments are not extracted, because they are not needed for the business questions.
- **Enterprise data warehouse (schema dw).** It holds the dimensional model: 7 dimension tables (Date, Customer, Product, Seller, Location, OrderStatus and PaymentType) and 2 fact tables (Fact_Sales at order-item level and Fact_OrderFulfilment at order level). Every dimension has a surrogate key and an Unknown member. The design is explained in Task 4.
- **Data marts (schema mart).** Two department views are built on the warehouse: the Sales Mart for sales and marketing (112,101 rows) and the Delivery and Satisfaction Mart for operations (96,476 rows). They are explained in Task 6.
- **ETL control and log (schema etl).** The tables run_log and validation_result record each validation run and its 16 checks. The stored procedure usp_run_validation fills them.

3.5 Presentation Layer

- **OLAP.** An SSAS multidimensional cube named Olist_Cube is built on the warehouse tables. It has the dimensions Date, Customer, Product, Seller, Order Status, Payment Type and Location (used twice, as customer location and seller location), user hierarchies for date, location, product and payment, two measure groups (sales and order fulfilment), and calculated measures such as revenue, average delivery days, late-delivery rate and average review score. In the cube browser the OLAP operations drill-down, roll-up, slice, dice and pivot are demonstrated (Task 7).
- **Reporting tool.** Power BI Desktop is used to build the report.
- **BI dashboards.** The report has three pages: an Executive Summary with KPIs and summary cards, a Trend Analysis with time-based and comparison charts, and an Interactive Analysis with slicers and drill-down (Task 7).

3.6 Data Flow Through the Architecture

1. **Extract.** Package 01 reads the database, the two CSV files and the holiday API.
2. **Stage.** The data is copied into the stg schema and is not yet cleaned. Row counts are compared with the sources.
3. **Transform.** SQL queries in packages 02 and 03 clean and reshape the data using the rules found by profiling.
4. **Load.** Dimensions are loaded first and facts second.
5. **Store.** The result is stored in the dw schema, and 16 validation checks confirm that it matches the source.
6. **Data marts.** The mart schema publishes department-specific views of the warehouse.
7. **Analyse.** The SSAS cube and Power BI read the warehouse and present hierarchies, measures, reports and dashboards to users.

3.7 Design Decisions

- **A staging area protects the source and allows reruns.** The operational database is read once. If a load fails, the process can be restarted from staging without reading the sources again.
- **Four schemas in one database.** Raw data (stg), clean analytical data (dw), department views (mart) and logs (etl) are kept apart but are easy to set up, back up and reproduce on another computer. A production system would normally use separate servers.
- **A dimensional model suits analysis.** Facts and dimensions make questions about time, product, location and customer easy to answer. Two fact tables are used because delivery, payment and review values belong to the whole order, and repeating them on every item row would distort averages.
- **A full load keeps the ETL simple and repeatable.** This fits a fixed snapshot of data.
- **Validation gives evidence.** Sixteen automated checks compare row counts and totals with the source after every run, and the results are stored.
- **The tools meet the assignment.** SQL Server, SSIS and T-SQL perform the data work, SSAS provides the OLAP layer, and Power BI provides the dashboards.

Task 4: Dimensional Data Warehouse Design and Implementation

4.1 Design Approach

The Olist data describes two business processes that operate at different levels of detail. Selling products is recorded at the level of an order item, while delivering an order and satisfying the customer are recorded at the level of the whole order. The model therefore uses two fact tables that share the same dimensions, a design known as a fact constellation (or galaxy schema).

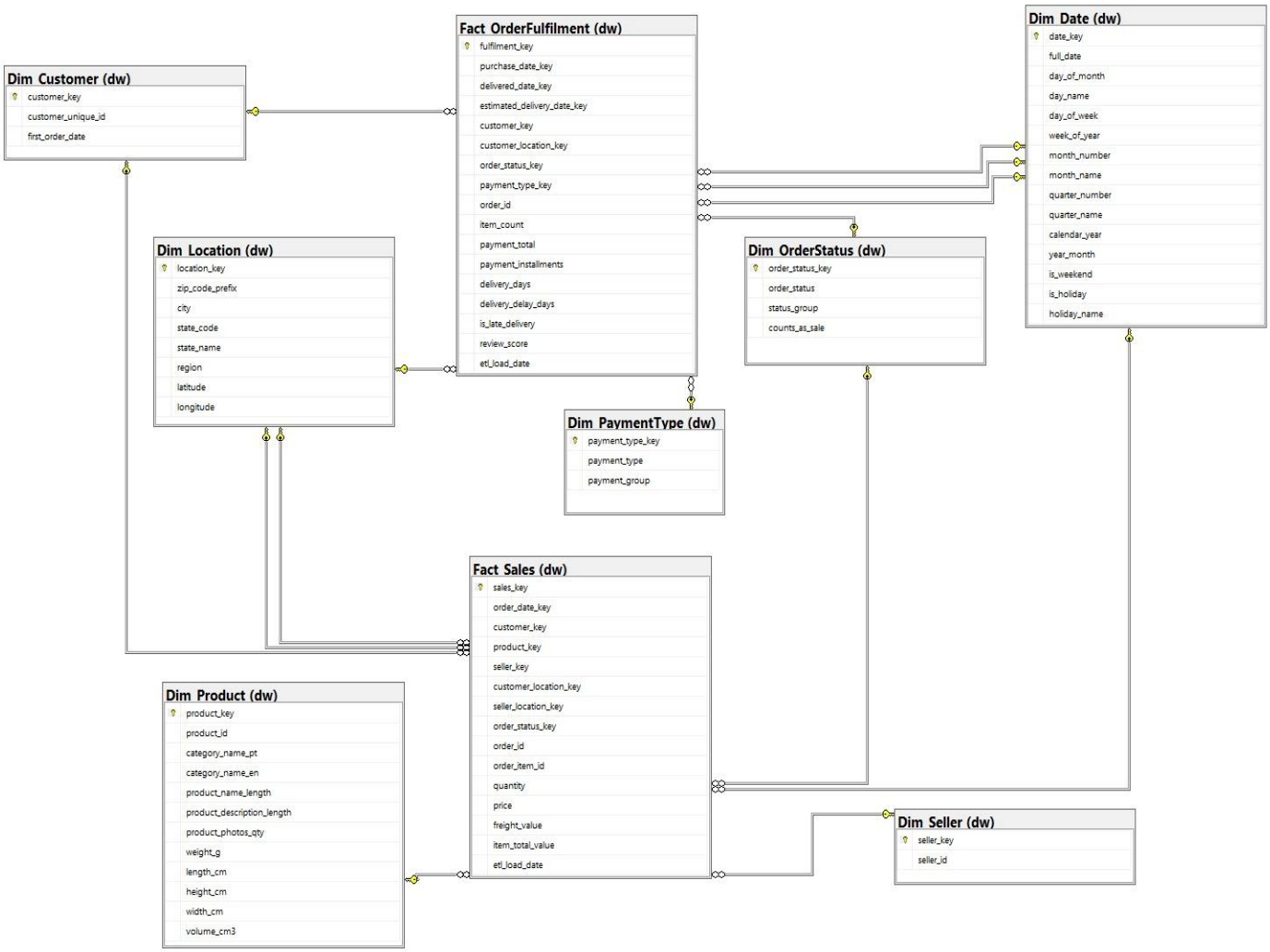
The two fact tables

Item	Fact_Sales	Fact_OrderFulfilment
Business process	Selling products through the marketplace	Delivering orders and satisfying customers
Grain	One row per order item (one product in one order)	One row per order
Measures	quantity, price, freight_value, item_total_value	item_count,payment_total,payment_installments, delivery_days,delivery_delay_days, is_late_delivery, review_score
Questions answered	Sales trends, best categories and sellers, regional sales	Delivery time, late deliveries, payment methods, review scores, holiday effects
Expected rows	112,650 (same as order_items)	99,441 (same as orders)

Why two fact tables rather than one? Every item in an order shares the same delivery date, payments and review. Storing these order-level values on each item row would count an order with three items three times, which would distort averages and totals. Separate fact tables at their natural grain avoid this problem. Because both use the same conformed dimensions, results from the two can still be compared directly.

4.2 Schema Diagram

Fact constellation schema of the Olist data warehouse



4.3 Fact Tables

Additive measures can be summed across any dimension.

Non-additive measures should be averaged or counted instead, because summing them gives no meaning.

Fact_Sales (grain: one row per order item)

Column	Data type	Role	Description
sales_key	BIGINT identity	Primary key	Surrogate key of the fact row
order_date_key	INT	Foreign key	Date the order was placed (Dim_Date)
customer_key	INT	Foreign key	Customer who bought (Dim_Customer)
product_key	INT	Foreign key	Product sold (Dim_Product)
seller_key	INT	Foreign key	Seller who sold it (Dim_Seller)
customer_location_key	INT	Foreign key	Location of the customer (Dim_Location)
seller_location_key	INT	Foreign key	Location of the seller (Dim_Location)
order_status_key	INT	Foreign key	Status of the order (Dim_OrderStatus)

Column	Data type	Role	Description
order_id	CHAR(32)	Degenerate dimension	Order number. It has no table of its own.
order_item_id	INT	Degenerate dimension	Item number inside the order. With order_id it defines the grain (unique constraint).
quantity	INT	Measure, additive	Always 1 per row. Summing gives the number of items sold.
price	DECIMAL(10,2)	Measure, additive	Item price in Brazilian reais
freight_value	DECIMAL(10,2)	Measure, additive	Shipping charge for the item
item_total_value	DECIMAL(10,2)	Measure, additive, derived	price + freight_value
etl_load_date	DATETIME2(0)	Audit	When the ETL inserted the row

Fact_OrderFulfilment (grain: one row per order)

Column	Data type	Role	Description
fulfilment_key	BIGINT identity	Primary key	Surrogate key of the fact row
purchase_date_key	INT	Foreign key	Date the order was placed (Dim_Date, role 1)
delivered_date_key	INT	Foreign key	Date delivered to the customer (Dim_Date, role 2). -1 if not delivered.
estimated_delivery_date_key	INT	Foreign key	Date promised to the customer (Dim_Date, role 3)
customer_key	INT	Foreign key	Customer (Dim_Customer)
customer_location_key	INT	Foreign key	Customer location (Dim_Location)
order_status_key	INT	Foreign key	Order status (Dim_OrderStatus)
payment_type_key	INT	Foreign key	Main payment type, the one with the largest value (Dim_PaymentType)
order_id	CHAR(32)	Degenerate dimension	Order number. Unique, so it defines the grain.
item_count	INT	Measure, additive	Number of items in the order
payment_total	DECIMAL(12,2)	Measure, additive	Sum of all payments for the order
payment_installments	INT	Measure, non-additive	Instalments of the main payment. Empty if there is no payment.

Column	Data type	Role	Description
delivery_days	INT	Measure, non-additive	Days from purchase to delivery. Empty if not delivered.
delivery_delay_days	INT	Measure, non-additive	Delivery date minus estimated date. Positive means late, negative means early.
is_late_delivery	BIT	Measure, additive	1 if late, 0 if on time. Summing gives the number of late orders. Empty if not delivered.
review_score	TINYINT	Measure, non-additive	Score from 1 to 5 of the latest review. Empty if the order has no review.
etl_load_date	DATETIME2(0)	Audit	When the ETL inserted the row

4.4 Dimension Tables

Dim_Date (one row per calendar day, 1 January 2016 to 31 December 2018)

Column	Data type	Description
date_key	INT	Primary key. The date as a number, for example 20171124. The value -1 means Unknown.
full_date	DATE	The calendar date
day_of_month	TINYINT	Day number, 1 to 31
day_name	VARCHAR(10)	Monday to Sunday
day_of_week	TINYINT	1 (Monday) to 7 (Sunday)
week_of_year	TINYINT	Week number in the year
month_number	TINYINT	1 to 12
month_name	VARCHAR(10)	January to December
quarter_number	TINYINT	1 to 4
quarter_name	VARCHAR(10)	Q1 to Q4
calendar_year	SMALLINT	Year, 2016 to 2018
year_month	VARCHAR(10)	Year and month for sorting, for example 2017-11
is_weekend	BIT	1 for Saturday or Sunday
is_holiday	BIT	1 for a Brazilian national public holiday (from the holiday API)
holiday_name	NVARCHAR(100)	Name of the holiday. Empty on other days.

Dim_Customer (one row per real customer)

Column	Data type	Description
customer_key	INT identity	Primary key (surrogate)
customer_unique_id	CHAR(32)	Business key. One value per real person, even if the person ordered many times.
first_order_date	DATE	Date of the first order of the customer in the data

Dim_Product (one row per product)

Column	Data type	Description
product_key	INT identity	Primary key (surrogate)
product_id	CHAR(32)	Business key
category_name_pt	NVARCHAR(100)	Category name in Portuguese, as in the source
category_name_en	NVARCHAR(100)	Category name in English. Unknown if the product has no category.
product_name_length	INT	Length of the product name (spelling of the source column corrected)
product_description_length	INT	Length of the product description
product_photos_qty	INT	Number of photos
weight_g, length_cm, height_cm, width_cm	INT	Weight and size of the product
volume_cm3	INT	Derived: length x height x width

Dim_Seller (one row per seller)

Column	Data type	Description
seller_key	INT identity	Primary key (surrogate)
seller_id	CHAR(32)	Business key

Dim_Location (one row per zip code prefix)

Column	Data type	Description
location_key	INT identity	Primary key (surrogate)
zip_code_prefix	VARCHAR(5)	Business key
city	NVARCHAR(100)	City name, standardised
state_code	CHAR(2)	Two-letter state code, for example SP
state_name	NVARCHAR(50)	Full state name, derived from the code
region	NVARCHAR(20)	Derived: North, Northeast, Central-West, Southeast or South
latitude, longitude	DECIMAL(9,6)	Average of the geolocation points for the zip prefix

Dim_OrderStatus (one row per order status)

Column	Data type	Description
order_status_key	INT identity	Primary key (surrogate)
order_status	VARCHAR(20)	Business key, the status in the source

Column	Data type	Description
status_group	VARCHAR(20)	Delivered, In progress or Not fulfilled
counts_as_sale	BIT	0 for canceled and unavailable orders, 1 for all others

Order status mapping

order_status	status_group	counts_as_sale
delivered	Delivered	1
shipped, invoiced, processing, approved, created	In progress	1
canceled, unavailable	Not fulfilled	0

Dim_PaymentType (one row per payment type)

Column	Data type	Description
payment_type_key	INT identity	Primary key (surrogate)
payment_type	VARCHAR(20)	Business key: credit_card, debit_card, boleto, voucher or not_defined
payment_group	VARCHAR(20)	Card, Bank slip, Voucher or Not defined

Payment type mapping

payment_type	payment_group	Note
credit_card, debit_card	Card	Paid by card
boleto	Bank slip	A Brazilian bank slip payment
voucher	Voucher	Paid with a voucher
not_defined	Not defined	3 payments with value 0 in the source

4.5 Relationships

Every relationship is many-to-one: many fact rows point to one dimension row. A dimension used more than once by a fact table is called a role-playing dimension.

Foreign key relationships

Fact table	Foreign key column	Dimension	Role
Fact_Sales	order_date_key	Dim_Date	Order date
Fact_Sales	customer_key	Dim_Customer	Buyer
Fact_Sales	product_key	Dim_Product	Product sold
Fact_Sales	seller_key	Dim_Seller	Seller
Fact_Sales	customer_location_key	Dim_Location	Customer location (role 1)
Fact_Sales	seller_location_key	Dim_Location	Seller location (role 2)

Fact table	Foreign key column	Dimension	Role
Fact_Sales	order_status_key	Dim_OrderStatus	Order status
Fact_OrderFulfilment	purchase_date_key	Dim_Date	Purchase date (role 1)
Fact_OrderFulfilment	delivered_date_key	Dim_Date	Delivery date (role 2)
Fact_OrderFulfilment	estimated_delivery_date_key	Dim_Date	Promised date (role 3)
Fact_OrderFulfilment	customer_key	Dim_Customer	Buyer
Fact_OrderFulfilment	customer_location_key	Dim_Location	Customer location
Fact_OrderFulfilment	order_status_key	Dim_OrderStatus	Order status
Fact_OrderFulfilment	payment_type_key	Dim_PaymentType	Main payment type

4.6 Hierarchies

Hierarchies let users drill down (from a summary to detail) and roll up (back to the summary) in the dashboard.

Dimension	Hierarchy (top to bottom)	Used for
Dim_Date	Year > Quarter > Month > Day	Trend analysis, from yearly totals down to single days
Dim_Location	Region > State > City > Zip prefix	Regional comparison, from a region down to a zip code
Dim_Product	Category > Product	Category performance, then single products
Dim_PaymentType	Payment group > Payment type	Card versus other methods, then credit versus debit
Dim_OrderStatus	Status group > Order status	Delivered versus not fulfilled orders
Dim_Customer, Dim_Seller	None	Olist anonymises them, so they hold only an identifier. They are analysed through location and the measures.

4.7 Modelling Techniques Used

- **Surrogate keys.** Each dimension has its own warehouse key, so the warehouse does not depend on the source identifiers.
- **Unknown members.** A row with key -1 in every dimension means no fact row ever needs a missing key.
- **Degenerate dimensions.** The order number is kept in the fact table because it has no attributes of its own.
- **Role-playing dimensions.** Dim_Date is used three times in one fact table, and Dim_Location is used for both customers and sellers. In the Power BI model the table is loaded once for each role.
- **Conformed dimensions**. Both fact tables use the same Dim_Date, Dim_Customer, Dim_Location and Dim_OrderStatus, so their results are comparable.
- **Slowly changing dimensions, Type 1.** Changed values are overwritten and no history is kept. The source has no change history, and the attributes are mostly stable.

4.8 Source-to-Target Summary

Where each warehouse table gets its data

Warehouse table	Source	Main logic
Dim_Date	Generated calendar and S3 holiday API	One row per day for 2016 to 2018. Holiday flag from the API.
Dim_Customer	customers and orders	One row per customer_unique_id. First order date from orders.
Dim_Product	products and S2 category translation	Add English category, handle missing values, derive volume.
Dim_Seller	sellers	One row per seller_id.
Dim_Location	S2 geolocation, customers and sellers	One row per zip prefix. Average coordinates. Derive state name and region.
Dim_OrderStatus	orders	Distinct statuses with the order status mapping in Section 4.4.
Dim_PaymentType	order_payments	Distinct payment types with the payment type mapping in Section 4.4.
Fact_Sales	order_items and orders	One row per item. Look up all dimension keys. Derive item_total_value.
Fact_OrderFulfilment	orders, order_items, order_payments and order_reviews	One row per order. Aggregate items and payments. Take the latest review. Calculate delivery measures.

4.9 Design Assumptions

1. **Revenue.** Revenue is the item price of orders where counts_as_sale = 1. Canceled and unavailable orders are excluded. Freight is reported separately.

2. **Currency.** All values are in Brazilian reais. No conversion or inflation adjustment is made.

3. **Customer identity.** A customer is identified by customer_unique_id. The customer location comes from each order, so it shows where the customer was at the time of the order.

4. **History.** Dimensions use Type 1 (overwrite). No history of changes is kept.

5. **Missing values.** Missing dimension values use the Unknown row (key -1). Missing measures stay empty, not zero.

6. **Reviews.** Only the latest review of each order is used. Review comment text is not loaded because it is not needed for the business questions.

7. **Payments.** The main payment type is the payment with the largest value in the order. If values are equal, the lowest payment sequence is used. payment_total adds all payments of the order.

8. **Delivery.** Delivery measures exist only for orders with a delivery date. An order is late if it is delivered on a later calendar day than the estimated date. The 8 delivered orders without a delivery date are treated as not delivered.

9. **Date range.** Dim_Date covers 1 January 2016 to 31 December 2018. Every date used in a fact table must exist in it, and this is checked in the ETL validation.

10. **Products.** The misspelled source columns (lenght) are corrected. The category is shown in English. If a category has no translation, the Portuguese name is used. Products with no category are Unknown.

11. **Locations.** There is one row per zip prefix. Coordinates are the average of all geolocation points for that prefix. The region is derived from the state code.

12. **Holidays.** The holiday flag covers national holidays only, because the API does not provide city-level holidays.

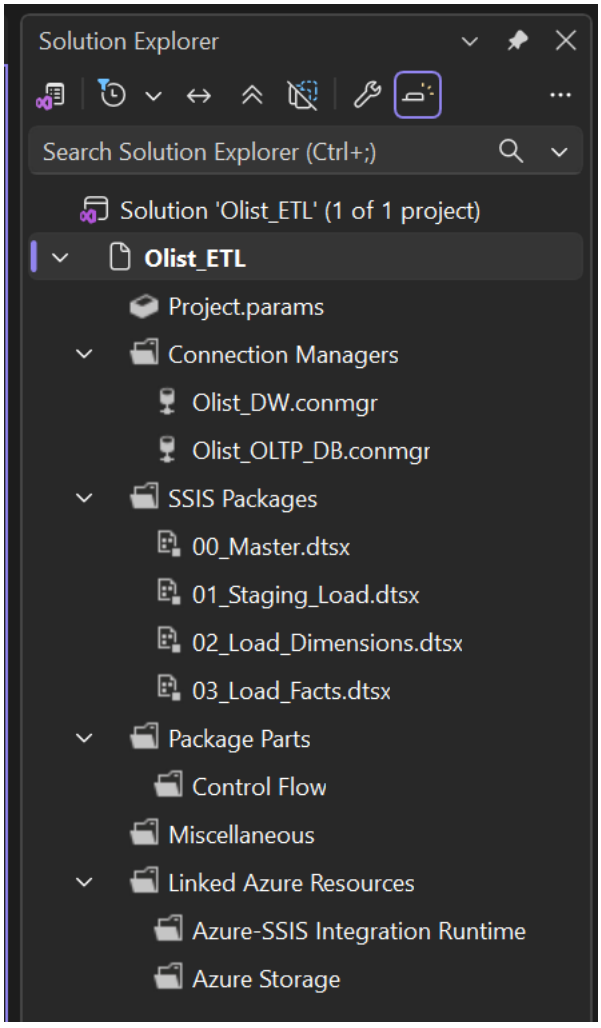
13. **Partial months.** The first and last months of the data contain very few orders. They are loaded, but dashboard analysis focuses on January 2017 to August 2018.

Task 5 – ETL Development

5.1 Overview

The ETL process was built with SQL Server Integration Services (SSIS) in one project, Olist_ETL. T-SQL is used inside the packages for source queries, transformations and validation. Four packages run in a fixed order under a master package.

Package	Purpose
01_Staging_Load	Extracts the three sources into the staging area
02_Load_Dimensions	Transforms and loads the 7 dimension tables
03_Load_Facts	Transforms and loads the 2 fact tables
00_Master	Runs 01, 02 and 03 in order, then the validation procedure

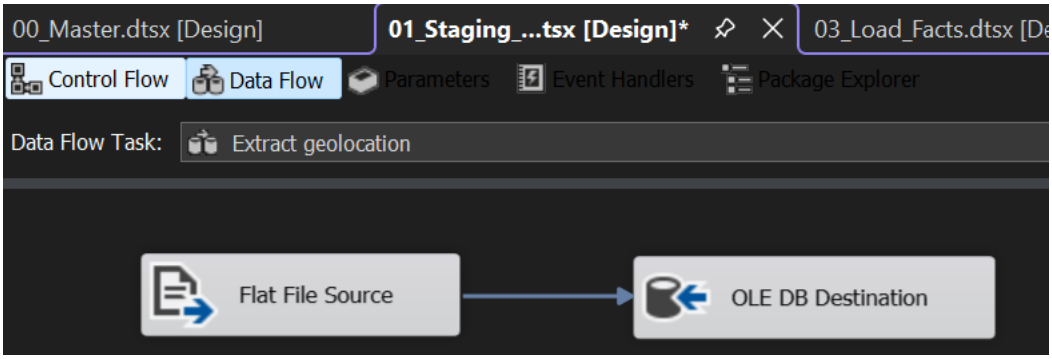


Solution Explorer of the Olist_ETL project

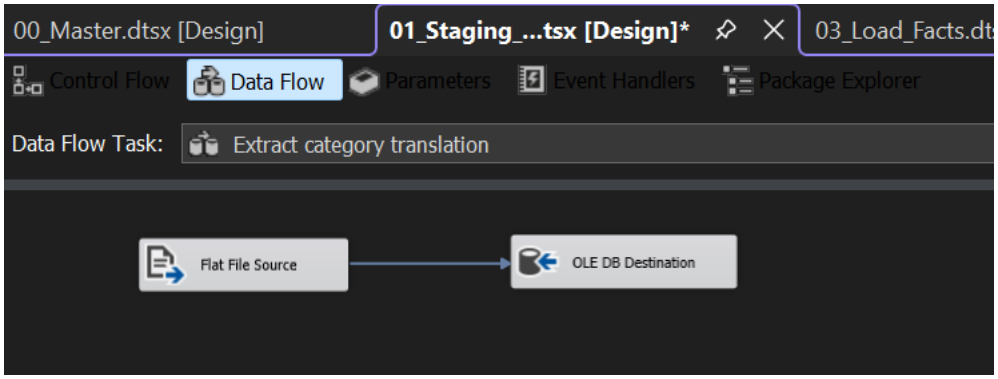
5.2 Extract

5.2.1 Extraction method and source connections

Source	Extraction method	Source connection
S1 Olist database	OLE DB Source to OLE DB Destination (7 data flows)	Project connection Olist_OLTP_DB: Microsoft OLE DB Driver for SQL Server, server localhost, Windows authentication, database Olist_OLTP_DB
S2 CSV files	Flat File Source to OLE DB Destination (2 data flows)	Flat File connections to the source folder: UTF-8 (code page 65001), text qualifier ", header row skipped, columns read as Unicode text
S3 Holiday API	Script Task (C#) plus an Execute SQL Task	HTTPS request to https://date.nager.at/api/v3/PublicHolidays/{year}/BR for 2016, 2017 and 2018



Extract Geolocation from flat files



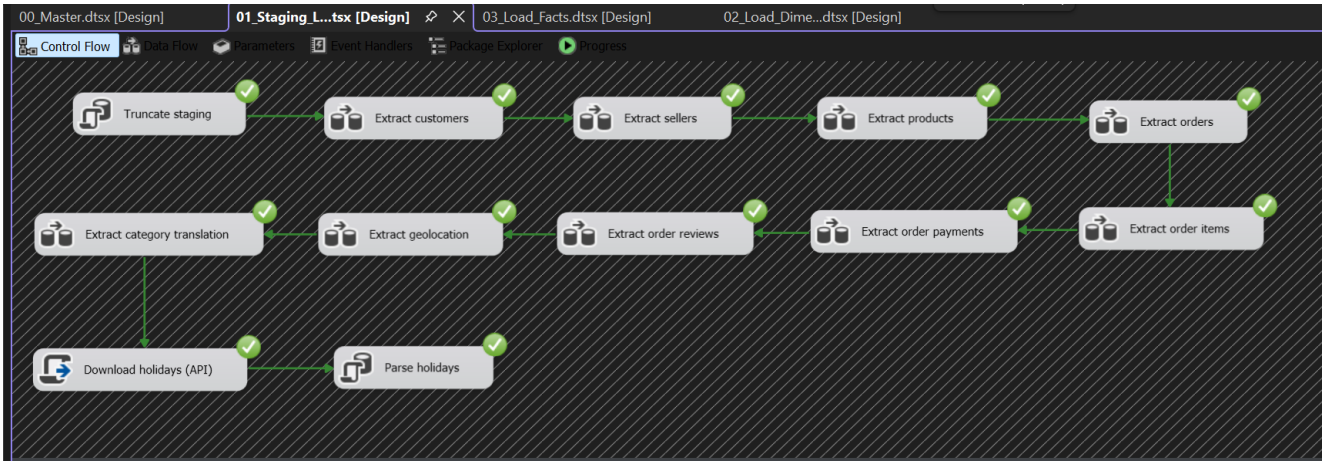
Extract category translation from flat files

5.2.2 Extraction process

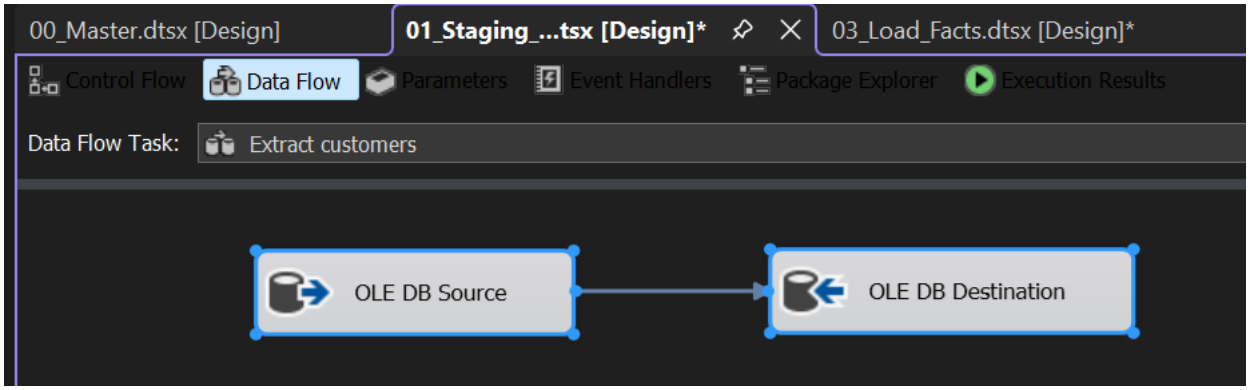
Package 01 runs these steps in order:

1. An Execute SQL Task, "Truncate staging", empties the ten staging tables so that each run starts clean.
2. Seven data flows copy customers, sellers, products, orders, order items, order payments and order reviews from Olist_OLTP_DB to the staging tables. For reviews, a SQL command selects only the columns needed, so the comment text is not extracted.
3. Two data flows load the geolocation file (1,000,163 rows) and the category translation file (71 rows).
4. A Script Task, "Download holidays (API)", requests the holiday data for each year, saves each response as a raw JSON file, and stores the text in a staging table. If the call fails, it writes a warning and uses the saved file, so the package does not fail because of the internet connection.

5. An Execute SQL Task, "Parse holidays", reads the JSON with OPENJSON into stg_public_holidays (42 rows, with the date, names, the nationwide flag and the holiday type).



Control flow of package 01



Extract Customers (OLE DB Source to OLE DB Destination)

ScriptMain.cs

ST_d9aa0742dc834c4192562b36a6017abb | ST_d9aa0742dc834c4192562b36a6017abb.ScriptMain | Main()

```
/// Before returning from this method, set the value of Dts.TaskResult to indicate success or failure.
/// To open Help, press F1.
/// </summary>
0 references
public void Main()
{
    ServicePointManager.SecurityProtocol = SecurityProtocolType.Tls12;
    string folder = @"C:\DWBI_Project\Holiday_JSON";
    string connStr = "Server=localhost;Database=Olist_DW;Integrated Security=true;TrustServerCertificate=true;";
    bool fireAgain = true;
    Directory.CreateDirectory(folder);

    using (SqlConnection conn = new SqlConnection(connStr))
    {
        conn.Open();
        new SqlCommand("DELETE FROM stg.stg_holidays_raw", conn).ExecuteNonQuery();

        for (int year = 2016; year <= 2018; year++)
        {
            string url = "https://date.nager.at/api/v3/PublicHolidays/" + year + "/BR";
            string file = Path.Combine(folder, "holidays_" + year + ".json");
            string json = null;
            try
            {
                using (WebClient client = new WebClient())
                {
                    client.Encoding = Encoding.UTF8;
                    json = client.DownloadString(url);
                }
                if (!json.TrimStart().StartsWith("[")) throw new Exception("Unexpected API response");
                File.WriteAllText(file, json, new UTF8Encoding(false));
                Dts.Events.FireInformation(0, "Holiday API", "Downloaded " + year + " from the API", "", 0, ref fireAgain);
            }
            catch (Exception ex)
            {
                Dts.Events.FireWarning(0, "Holiday API", "API call failed for " + year + " (" + ex.Message + "). Using the saved file.", "", 0);
                if (!File.Exists(file))
                {
                    Dts.Events.FireError(0, "Holiday API", "No saved file for " + year, "", 0);
                    Dts.TaskResult = (int)ScriptResults.Failure;
                    return;
                }
                json = File.ReadAllText(file, Encoding.UTF8);
            }

            SqlCommand cmd = new SqlCommand("INSERT INTO stg.stg_holidays_raw (holiday_year, raw_json) VALUES (@y, @j)", conn);
            cmd.Parameters.AddWithValue("@y", year);
            cmd.Parameters.Add("@j", SqlDbType.NVarChar, -1).Value = json;
            cmd.ExecuteNonQuery();
        }
    }
    Dts.TaskResult = (int)ScriptResults.Success;
}
```

Script task code for download holidays from API

```
SELECT TOP 5 * FROM stg.stg_public_holidays;
```

88 % | No issues found

Results | Messages

	holiday_date	local_name	english_name	country_code	is_fixed	is_global	launch_year	holiday_types	counties
1	2016-01-01	Confraternização Universal	New Year's Day	BR	0	1	NULL	["Public"]	NULL
2	2016-02-08	Carnaval	Carnival	BR	0	1	NULL	["Bank","Optional"]	NULL
3	2016-02-09	Carnaval	Carnival	BR	0	1	NULL	["Bank","Optional"]	NULL
4	2016-03-25	Sexta-feira Santa	Good Friday	BR	0	1	NULL	["Public"]	NULL
5	2016-03-27	Domingo de Páscoa	Easter Sunday	BR	0	1	NULL	["Public"]	NULL

Holidays staging table in SSMS after downloading holidays from the API

5.3 Transform

The transformations were designed from the profiling results (Task 2, Table 2.7). They are written as T-SQL queries that serve as the sources of the data flows in packages 02 and 03.

Transformation type	What was done	Evidence
Missing value handling	Products without a category become Unknown. Orders not delivered keep empty delivery measures and use the Unknown date key (-1). Orders without a review keep an empty score. One order without a payment uses the	610 products without category, 2,965 orders without a delivery date, 768 orders without a review

	Unknown payment type. Locations without coordinates keep empty coordinates.	
Duplicate removal	One customer row per customer_unique_id. Geolocation points reduced to one row per zip prefix. Only the latest review of each order is kept. One main payment type per order.	99,441 customer rows became 96,096 customers. 1,000,163 geolocation rows became 15,078 locations.
Data cleansing	42 geolocation points outside Brazil were removed before averaging. State-level holidays are not flagged. Canceled and unavailable orders are marked as not counting as sales.	42 points removed. 3 of 42 holidays not nationwide. 625 canceled and 609 unavailable orders.
Standardisation	Misspelled source columns corrected (lenght). English category names used, with the Portuguese name as a fallback. City names trimmed and state codes in upper case. Order statuses and payment types grouped. Zip codes kept as 5-character text.	Dim_Product, Dim_Location, Dim_OrderStatus, Dim_PaymentType
Data type conversion	Text latitude and longitude converted to numbers and then to DECIMAL(9,6). Dates converted to an integer date key (yyyymmdd). Timestamps converted to dates for key matching. JSON text converted to typed columns (date, true or false, number) with OPENJSON.	Dim_Location, Dim_Date, stg_public_holidays
Aggregation	Items counted and payments summed for each order. The earliest order date found for each customer. Coordinates averaged for each zip prefix.	item_count, payment_total, first_order_date, latitude and longitude
Derived attributes	item_total_value, volume_cm3, delivery_days, delivery_delay_days, is_late_delivery, state_name, region, status_group, counts_as_sale, payment_group, holiday flag and name, week, quarter, year_month, is_weekend	Dim_Date, Dim_Location, Dim_Product, both fact tables

5.4 Load

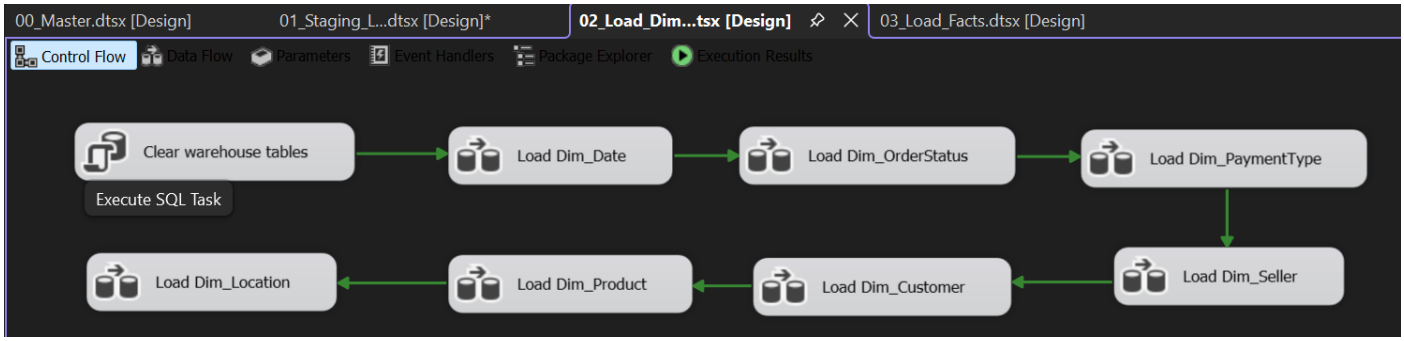
5.4.1 Loading sequence

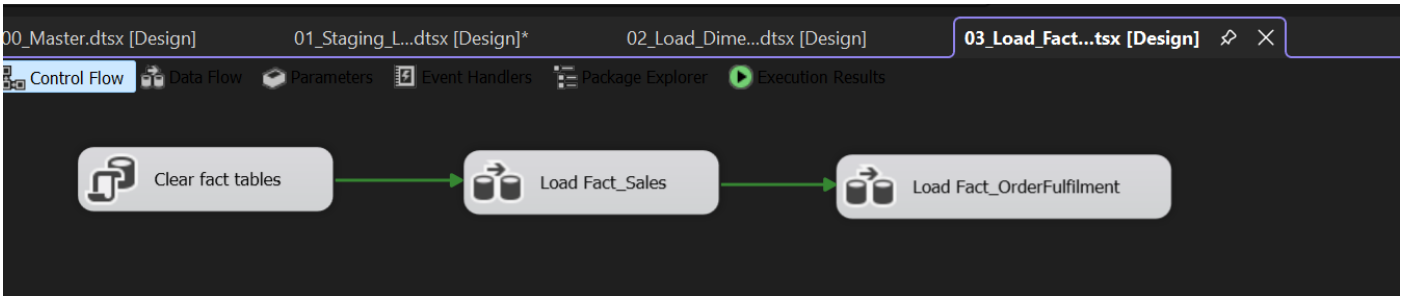
Step	Package	Task	Target	Rows
1	02	Clear warehouse tables	Facts, then dimensions (the Unknown rows are kept)	
2	02	Load Dim_Date	dw.Dim_Date	1,097
3	02	Load Dim_OrderStatus	dw.Dim_OrderStatus	9
4	02	Load Dim_PaymentType	dw.Dim_PaymentType	6
5	02	Load Dim_Seller	dw.Dim_Seller	3,096
6	02	Load Dim_Customer	dw.Dim_Customer	96,097
7	02	Load Dim_Product	dw.Dim_Product	32,952
8	02	Load Dim_Location	dw.Dim_Location	15,079
9	03	Clear fact tables	Facts, with identity reset	
10	03	Load Fact_Sales	dw.Fact_Sales	112,650
11	03	Load Fact_OrderFulfilment	dw.Fact_OrderFulfilment	99,441

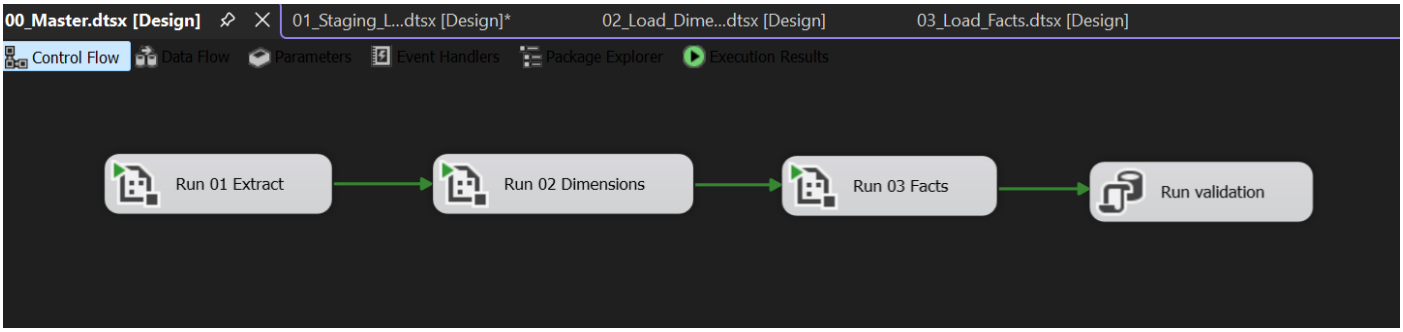
Dimension rows include the Unknown row. The dimensions are loaded before the facts because each fact row needs the surrogate keys of its dimensions. The facts are cleared before the dimensions because they depend on them through foreign keys. The destinations use fast load, the key columns are left to SQL Server to generate, and the Keep identity option is off.

5.4.2 ETL workflow

The master package 00_Master runs package 01, then 02, then 03, then an Execute SQL Task that calls the validation procedure. If a task fails, the following tasks do not run. Running the master package twice from a clean state produced identical results.







5.5 Validation Results

#	Check	Expected	Actual	Status
1	Staging orders = source orders	99,441	99,441	PASS
2	Staging order items = source order items	112,650	112,650	PASS
3	Staging customers = source customers	99,441	99,441	PASS
4	Fact_Sales rows = staged order items	112,650	112,650	PASS
5	Fact_OrderFulfilment rows = staged orders	99,441	99,441	PASS
6	Dim_Customer = distinct customers + Unknown	96,097	96,097	PASS
7	Dim_Product = staged products + Unknown	32,952	32,952	PASS

8	Total item price = staging	13,591,643.70	13,591,643.70	PASS
9	Total freight = staging	2,251,909.54	2,251,909.54	PASS
10	Total payments = staging	16,008,872.12	16,008,872.12	PASS
11	Fact_Sales rows with an Unknown key	0	0	PASS
12	Fact_OrderFulfilment rows with an Unknown key	0	0	PASS
13	Review scores outside 1 to 5	0	0	PASS
14	Negative delivery days	0	0	PASS
15	Late flag on undelivered orders	0	0	PASS
16	Duplicate order items in Fact_Sales	0	0	PASS

5.6 Problems Found and How They Were Solved

Problem	Cause	Solution
The reviews file failed to import (OLE DB provider error)	A fixed line-ending option did not match the file	Removed the option so SQL Server detected the line endings
72 rows in the translation table, columns named "column 1" and "column 2"	The header row was loaded as data	Rebuilt the load in SSIS with the header row skipped (71 rows)
Accents garbled in the holiday names	The JSON was read with the wrong encoding	Reloaded as UTF-8
Conversion error between Unicode and non-Unicode on customer_city	The source and staging columns had different types	Recreated the staging tables with matching Unicode types
Staging was first loaded with T-SQL	The assignment requires SSIS for the ETL	Rebuilt the extraction as an SSIS package
Validation procedure failed with "Invalid object name"	The source database name in the script was wrong	Corrected the database name in the procedure

Task 6: Data Mart Development

6.1 Overview

Two department-specific data marts were created on top of the enterprise data warehouse. A data mart is a smaller, focused view of the warehouse that gives one department exactly the data it needs, with that department's business rules already applied. Both marts are SQL views in the mart schema of the Olist_DW database. They are built from the warehouse tables, so they always show current warehouse data and need no extra ETL step.

	Sales Mart	Delivery and Satisfaction Mart
Object name	**mart.Sales_Mart**	**mart.Delivery_Satisfaction_Mart**
Grain	**One row per order item**	**One row per order**
Built from	**Fact_Sales and six dimensions**	**Fact_OrderFulfilment and five dimensions (Date three times)**
Business rule	**Only real sales: items of canceled and unavailable orders are removed**	**Only delivered orders: orders that have a delivery date**
Rows	**112,101**	**96,476**

6.2 Sales Mart

Purpose. It gives one flat table of real sales. Each row is one sold item, with the date, product category, seller, customer location, status and the money values already attached.

Target users. Sales and marketing managers and analysts.

Business rule applied. Only orders with counts_as_sale = 1 are included. This removes 549 items that belong to canceled and unavailable orders (112,650 items in the warehouse, 112,101 in the mart), so revenue taken from the mart is real revenue.

Columns.

- When: order_date, calendar_year, quarter_name, month_number, month_name, year_month, day_name, is_weekend, is_holiday, holiday_name
- What: product_id, product_category (English name)
- Who sold: seller_id, seller_state, seller_region
- Who bought: customer_unique_id, customer_city, customer_state, customer_region
- Status: order_status
- Measures: quantity, price, freight_value, item_total_value

Analytical benefits.

- Sales questions are answered from one table, with no joins and no knowledge of the warehouse keys.
- Revenue is correct by default, because the sales rule is built in.
- The same table supports questions on categories, sellers, regions, months and holidays, for example which categories earn the most, which regions buy the most, and whether holidays change sales.

6.3 Delivery and Satisfaction Mart

Purpose. It gives one flat table of delivered orders, with the delivery time, whether the order was late, the payment details and the customer's review score.

Target users. Operations and customer service managers.

Business rule applied. Only orders with a delivery date are included (99,441 orders minus 2,965 that were never delivered, which leaves 96,476). Delivery measures are therefore meaningful in every row.

Columns.

- When: purchase_date, purchase_year, purchase_quarter, purchase_month_number, purchase_month, purchase_year_month, delivered_date, estimated_delivery_date
- Where: customer_city, customer_state, customer_region
- Status and payment: order_status, status_group, payment_type, payment_group
- Measures: item_count, payment_total, payment_installments, delivery_days, delivery_delay_days, is_late_delivery, review_score

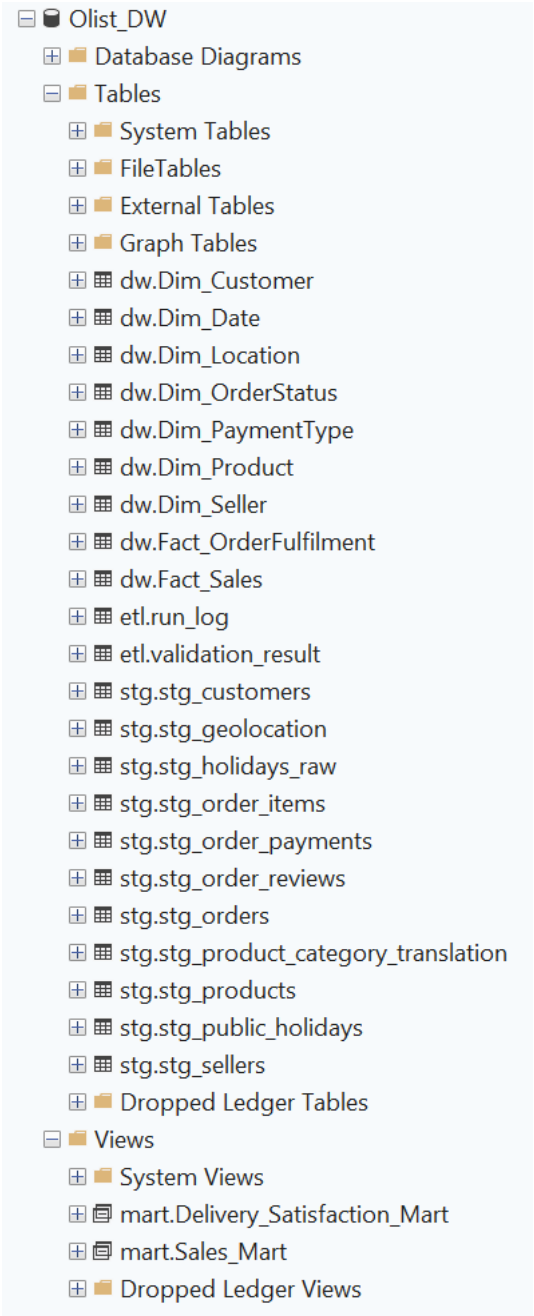
Analytical benefits.

- Delivery performance and customer satisfaction can be analysed together in one table.
- Averages of delivery time and review score are counted once per order, with no risk of double counting.
- It supports questions on late deliveries by region and month, on payment methods, and on whether late orders receive lower review scores.

6.4 Implementation

The marts were created with the script 06_data_marts.sql. Each mart is a view that joins a fact table to its dimensions with INNER JOINs, selects readable columns, and applies the filter of its department. The script ends with a check that compares the number of rows in each mart with the expected number taken from the warehouse (112,101 and 96,476). Both checks matched.

Design choice. Views were used instead of separate tables. A view always shows the current warehouse data and needs no extra loading step. With about 100,000 rows the speed is not a problem. For a much larger warehouse, the marts could be stored as tables and refreshed by the ETL.


Olist_DW
Database Diagrams
Tables
System Tables
FileTables
External Tables
Graph Tables
dw.Dim_Customer
dw.Dim_Date
dw.Dim_Location
dw.Dim_OrderStatus
dw.Dim_PaymentType
dw.Dim_Product
dw.Dim_Seller
dw.Fact_OrderFulfilment
dw.Fact_Sales
etl.run_log
etl.validation_result
stg.stg_customers
stg.stg_geolocation
stg.stg_holidays_raw
stg.stg_order_items
stg.stg_order_payments
stg.stg_order_reviews
stg.stg_orders
stg.stg_product_category_translation
stg.stg_products
stg.stg_public_holidays
stg.stg_sellers
Dropped Ledger Tables
Views
System Views
mart.Delivery_Satisfaction_Mart
mart.Sales_Mart
Dropped Ledger Views

Task 7: OLAP Analysis and Business Intelligence Dashboard Development

7.1 Overview

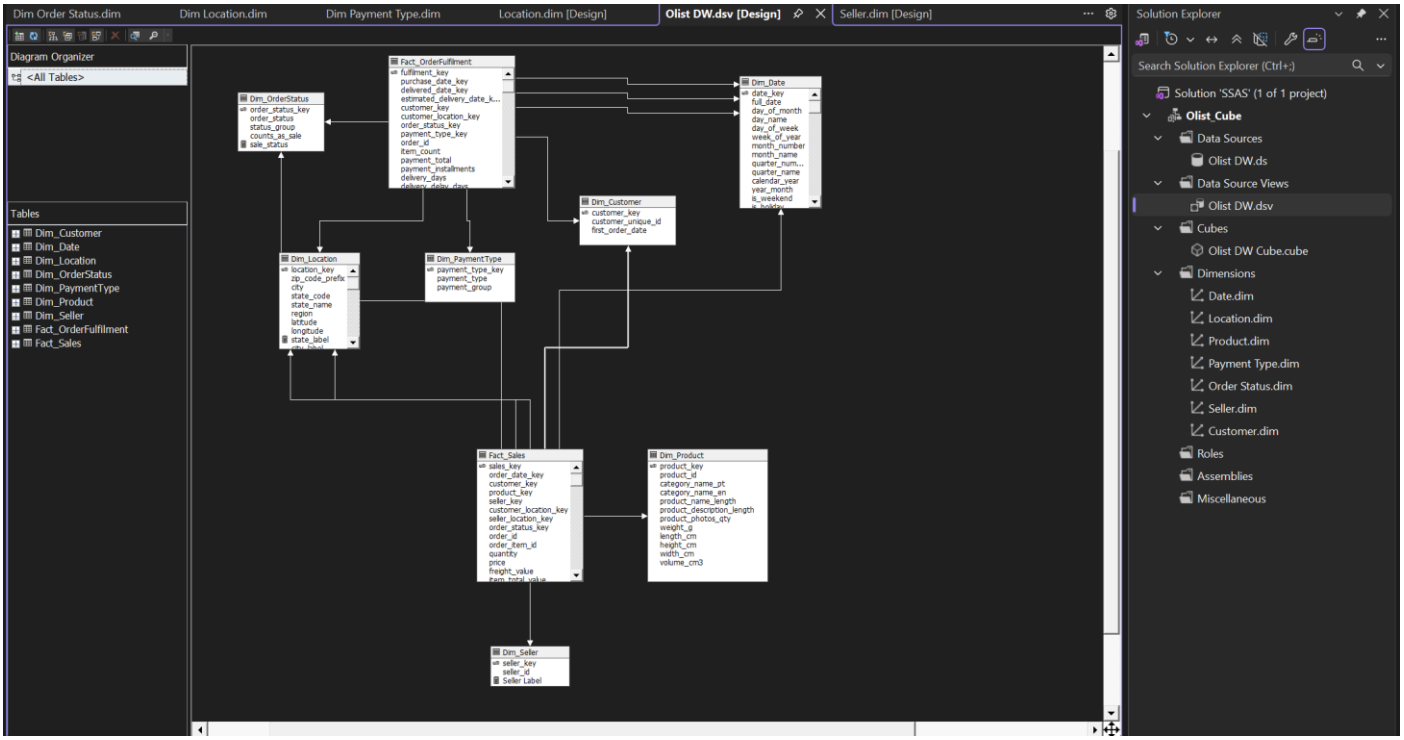
The analysis layer has two parts. An SSAS multidimensional cube (Olist_Cube) provides the OLAP model, and a three-page Power BI report is connected live to the cube. Both read the data warehouse Olist_DW, so the totals agree.

7.2 OLAP Cube Design

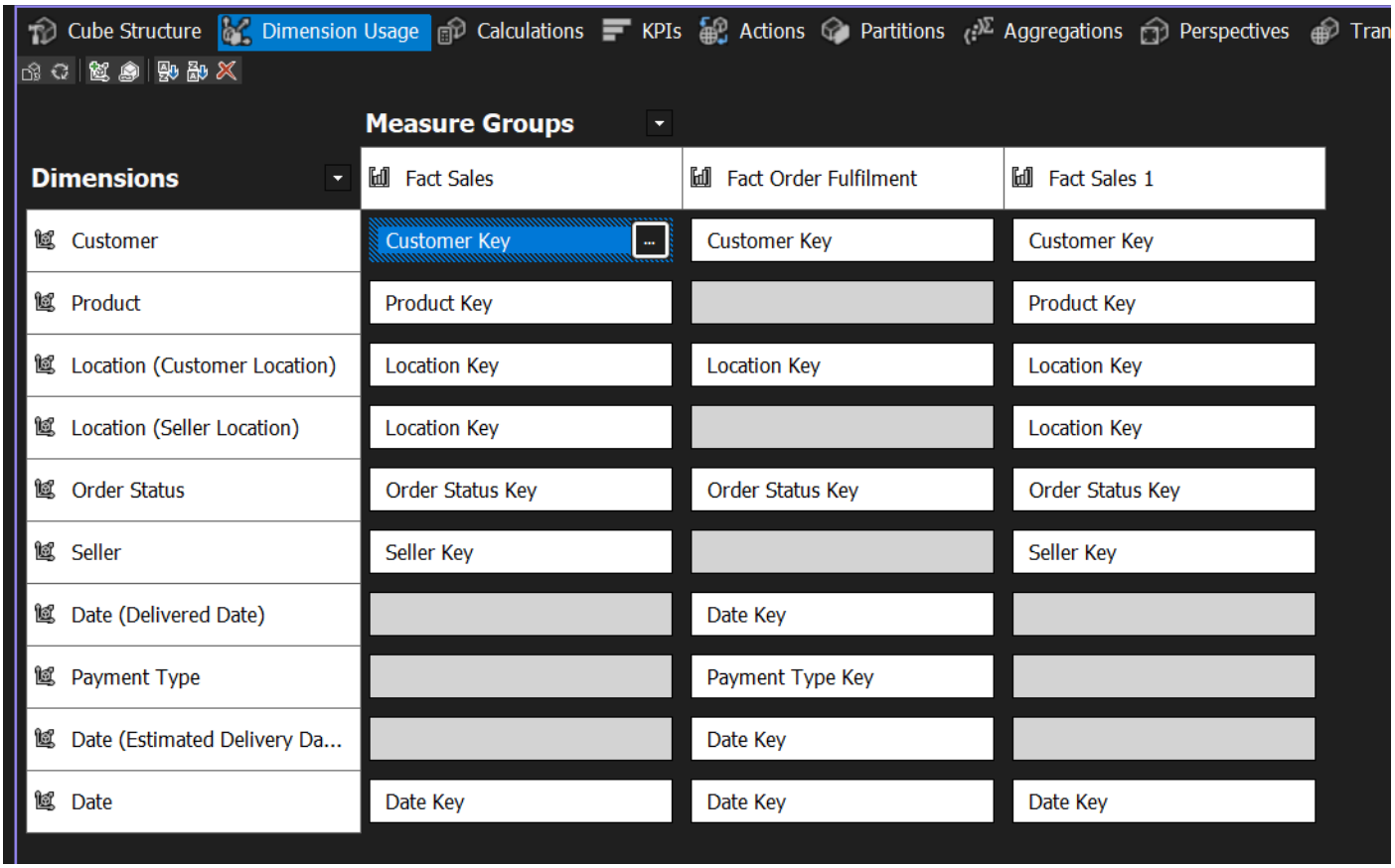
- **Data source and view.** The cube reads Olist_DW through an OLE DB data source. The data source view contains the nine warehouse tables. Named calculations add readable labels (for example year-quarter and state-city) and helper columns that a cube can add up (for example late order as 0 or 1).
- **Dimensions.** Date, Customer, Product, Seller, Order Status and Payment Type, plus Location used twice (Customer Location and Seller Location). Date is also used as Delivered Date and Estimated Delivery Date for the order fulfilment facts.
- **Hierarchies.** Calendar (Year, Quarter, Month, Date), Geography (Region, State, City, Zip prefix), Product (Category, Product), Payment (Group, Type) and Status (Group, Status).
- **Measure groups.** Fact Sales (price, freight, quantity) and Fact Order Fulfilment (item count, payment total, delivery days, review score, delivered orders, review count, late orders).
- **Calculated members.**

Measure	Definition
Revenue	Price minus the price of the Not fulfilled status group
Freight, Items Sold, Sales Orders	The same rule applied to freight, quantity and orders with items
Avg Delivery Days	Delivery days divided by delivered orders
Avg Review Score	Review score divided by review count
Late Delivery %	Late orders divided by delivered orders

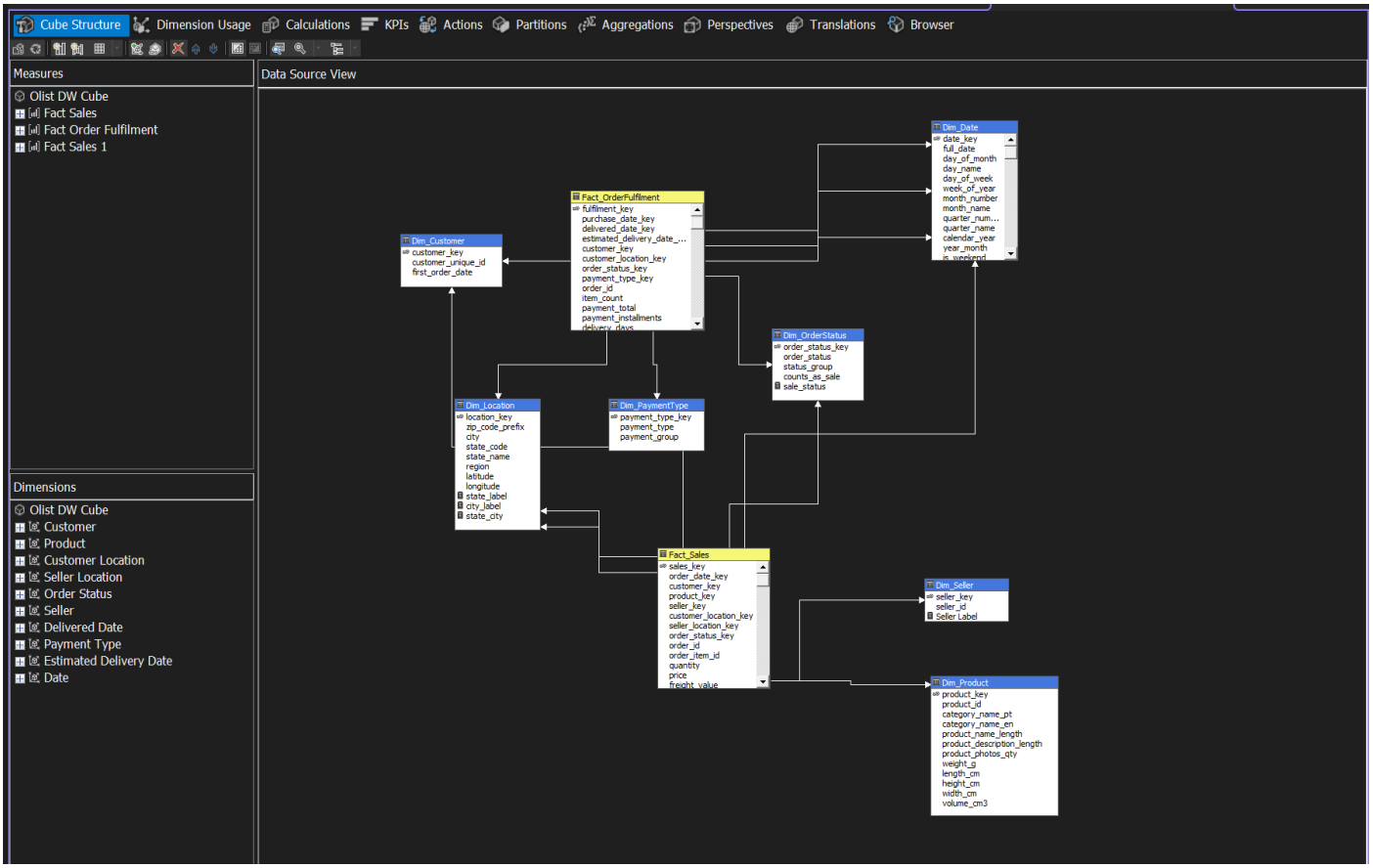
- **Rule for combining measures.** Sales measures work only with Date, Customer, Product, Seller, Customer Location, Seller Location and Order Status. Order measures work only with Date, Customer, Customer Location, Order Status and Payment Type. A measure used with an unrelated dimension repeats the same total on every row, so the dashboard avoids these combinations.



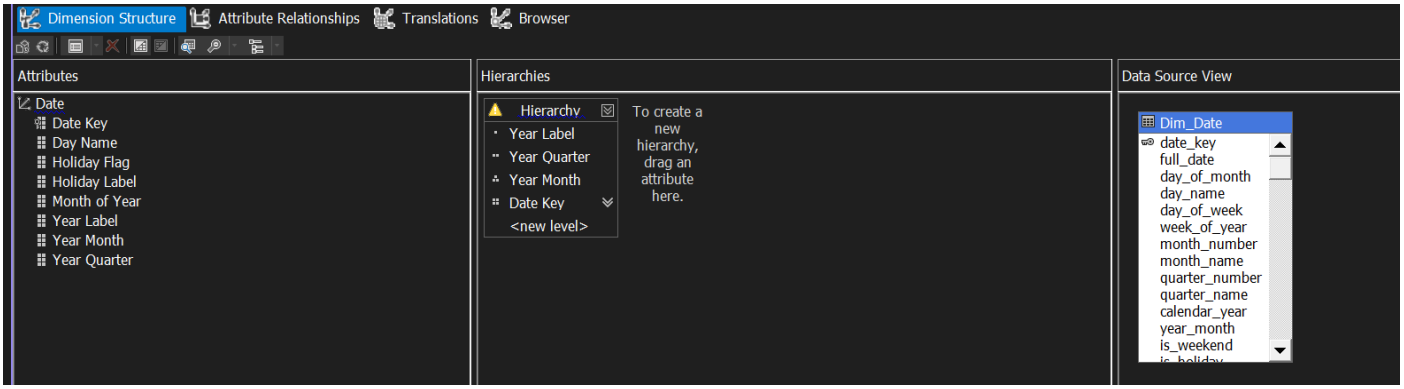
[Data Source View diagram]



[Dimension Usage tab]



[Cube Structure tab]

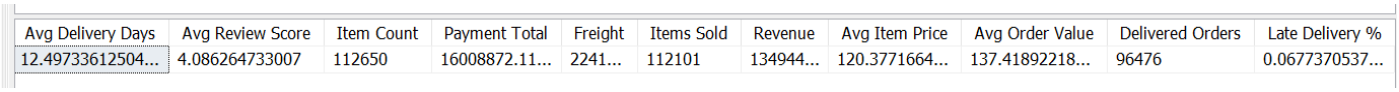


[Date dimension with the Calendar hierarchy]

7.3 Cube Verification

Measure	Cube result	Check
Price (no filter)	13,591,643.70	Equals the staging total
Quantity	112,650	Equals the order items
Fact Order Fulfilment Count	99,441	Equals the orders
Delivered Orders	96,476	Orders with a delivery date
Avg Delivery Days	12.50	1,205,693 days over 96,476 orders
Avg Review Score	4.09	403,204 over 98,673 reviews
Revenue (sales rule)	13,494,400.74	Equals Power BI

Items Sold	112,101	Equals Power BI
Sales Orders	98,199	Equals Power BI
Freight	2,241,126.29	Equals the warehouse



Avg Delivery Days	Avg Review Score	Item Count	Payment Total	Freight	Items Sold	Revenue	Avg Item Price	Avg Order Value	Delivered Orders	Late Delivery %
12.49733612504...	4.086264733007	112650	16008872.11...	2241...	112101	134944...	120.3771664...	137.41892218...	96476	0.0677370537...

[cube browser with the totals]

7.4 Power BI Report Design

The report connects live to Olist_Cube, so every measure comes from the cube and matches its numbers. All pages share one theme, a left sidebar with page navigation and slicers, and a title area. The sidebar slicers are Year, State City and Product Category. The month filter excludes the five nearly empty months (September, October and December 2016, and September and October 2018), so every page covers January 2017 to August 2018. Seller names are anonymised in the data, so sellers appear as numbers assigned by the warehouse.

Page 1: Executive Summary

Visual	Fields	Purpose
7 KPI cards	Revenue, Sales Orders, Items Sold, Avg Delivery Days, Avg Review Score, Late Delivery %, Freight	Overall performance at a glance
Revenue by category	Product category, Revenue (top 10)	Where revenue comes from
Revenue by seller	Seller, Revenue (top 10)	Seller concentration
Sales orders by state	Customer state (drills down to payment group), Sales Orders	Where orders come from and how they are paid
Sales orders by quarter	Year > Quarter > Month, Sales Orders	Order volume over time

Page 2: Trend Analysis

Visual	Fields	Purpose
Orders and items sold	Year > Quarter > Month, Sales Orders, Items Sold	Growth and seasonality
2017 versus 2018	Quarter or month of year, year as legend, Revenue	A comparison chart
Delivery trend	Year Month, Avg Delivery Days, Late Delivery %	Delivery speed over time
Satisfaction trend	Year Month, Avg Review Score	Customer satisfaction over time

Page 3: Interactive Analysis

Visual	Fields	Purpose
Revenue by seller region	Seller region, Revenue	Where sales are shipped from
Avg order value by region	Customer region, Avg Order Value	Spending per order
Review score by city	Customer state and city, Avg Review Score	Spread of satisfaction across cities
Drill-down matrix	Location hierarchy (State > City > Zip prefix), Revenue, Sales Orders, Avg Delivery Days, Late Delivery %, Avg Review Score	Drill-down and roll-up

Filters, slicers, drill-down and roll-up. The three slicers filter the pages. Year and State City filter every visual. The Category slicer filters only the revenue visuals, because the order measures have no product link. The location matrix and the date charts support drill-down, and the up arrow rolls back up.

7.5 Problems Solved in This Task

Problem	Cause	Solution
Login failed when processing the cube	The SSAS service account had no rights on Olist_DW	Gave the account read access to the database
Averages were thousands of days	Count measures were created as distinct counts	Replaced them with count of non-empty values
Revenue showed #Error	A formula used a different dimension name from the cube	Renamed the dimension to match
Extra measure groups appeared	Leftover measures from the first attempt	Deleted the empty groups
Seller IDs unreadable in charts	Source identifiers are anonymised hash codes	Used numbered labels, with the key reset in the ETL

olist

Sales & Delivery Analytics

PAGES

Executive Summary

Trend Analysis

Interactive Analysis

FILTERS

Year Label: All

State City: All

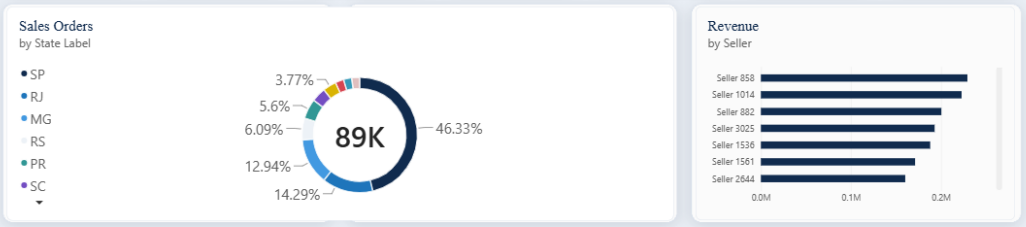
Category Name En: All

Source: Olist_DW · SSAS cube

Executive Summary

Olist Brazilian E-Commerce · 2016 – 2018

Revenue	Sales Orders	Items Sold	Avg Delivery Days	Avg Review Score	Late Delivery %	Freight
13.45M	98K	112K	12.5	4.09	6.79 %	2.23M





olist

Sales & Delivery Analytics

PAGES

Executive Summary

Trend Analysis

Interactive Analysis

FILTERS

Year Label: All

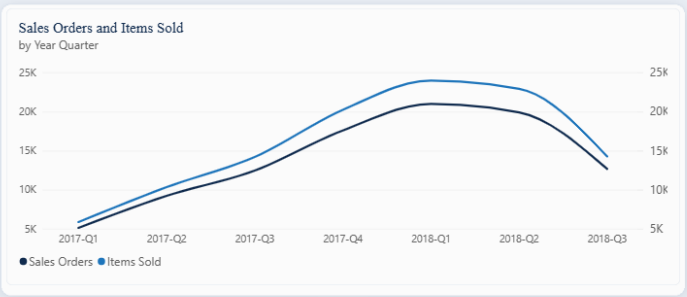
State City: All

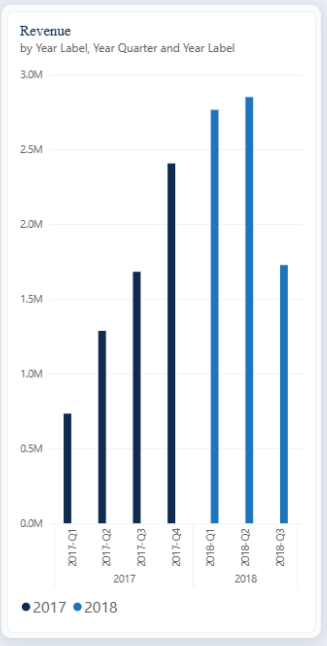
Category Name En: All

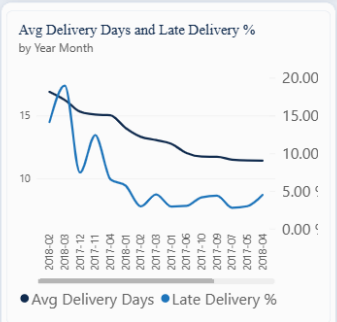
Source: Olist_DW · SSAS cube

Trend Analysis

Olist Brazilian E-Commerce · 2016 – 2018











Task 8: Business Insights and Recommendations

8.1 Insights

<u>Insight 1: Demand grew about fourfold within a year.</u>

Evidence:
Orders rose from about 5,100 in 2017-Q1 to about 20,900 in 2018-Q1. Quarterly revenue rose from about 0.73M to 2.76M, 3.8 times as much. Full-year 2017 revenue was 6.11M from 44,375 orders. In only eight months of 2018, revenue reached 7.34M from 53,531 orders.

Decision it supports:
The marketplace is scaling quickly, so logistics, support and seller capacity must grow with it.

Recommendation:
Plan carrier and staffing capacity against the 2018 run rate, not against 2017.

Insight 2: Growth levelled off in 2018, and November 2017 was the biggest spike.

Evidence:

Quarterly revenue grew about 15% from 2017-Q4 (about 2.40M) to 2018-Q1 (about 2.76M), then only about 3% to 2018-Q2 (about 2.84M). Orders slipped slightly, from about 20,900 to 19,900. The 2018-Q3 drop is a partial quarter, because it holds only July and August. In the Task 2 profiling, monthly orders stayed between about 6,200 and 7,300 throughout 2018. November 2017 had 7,544 orders, 63% above October (4631), and higher than any month in 2018.

Decision it supports:
Further growth will need new demand, not just more of the same, and peak seasons need early preparation.

Recommendation:
Prepare stock, promotions and delivery capacity before November, and look for new customer segments to restart growth.

Insight 3: Demand is concentrated in three states.

Evidence:
In 2017, São Paulo alone produced 2.19M of 6.11M revenue (35.9%) and 17,423 of 44,375 orders (39.3%). With Rio de Janeiro (14.8%) and Minas Gerais (11.8%), the three states produced 62.4% of revenue. The top five states (adding Rio Grande do Sul and Paraná) produced 73%.

Decision it supports:
Marketing and delivery partnerships are concentrated in a few places.

Recommendation:
Protect delivery quality in these three states, and run a targeted campaign in mid sized states such as BA, SC and GO.

Insight 4: Sellers are even more concentrated than customers.

Evidence:
In 2017, sellers in the Southeast generated 77.9% of revenue (4.76M), while customers in the Southeast (SP, RJ, MG and ES) accounted for 64.3% (3.93M). About 0.83M, or 13.6% of revenue, was therefore shipped from Southeast sellers to customers elsewhere. The North's sellers generated only 0.1%.

Decision it supports: Seller recruitment and shipping distance.

Recommendation: Recruit sellers in the South, Northeast and Central West, so products ship from closer to the buyers.

Insight 5: Customers in remote regions spend more per order.

Evidence:
In 2017, the average order value was about 178 in the North, 162 in the Northeast, 156 in the Central West, 134 in the South and 130 in the Southeast.

Decision it supports*:* Pricing, shipping and promotion by region.

Recommendation: Test free shipping thresholds and bundles in the Southeast and South, where orders are smaller. Check whether remote customers order less often but buy more each time.

Insight 6: Delivery slowdowns coincide with lower customer satisfaction.

Evidence:
The overall average delivery time is 12.5 days and 6.79% of deliveries are late. The months with the longest delivery times were February 2018 (about 17 days), March 2018, December 2017 and November 2017 (about 15 to 16 days). The average review score fell to about 3.9 in November 2017 and to about 3.75 in March 2018, against 4.09 overall, then recovered to about 4.3 by June 2018. These dips follow the two busiest periods.

Decision it supports:
Operations capacity during peak periods.

Recommendation:
Add carrier capacity before peak periods, set realistic delivery promises when volume is high, and warn customers when an order is running late.

Insight 7: Revenue depends on a few categories, but not on a few sellers.

Evidence:
The top five categories (health_beauty, watches_gifts, bed_bath_table, sports_leisure and computers_accessories) bring in roughly 5.3M, about 40% of revenue. The top ten bring in about 8.3M, about 62%. The largest seller earns about 0.23M, around 2% of revenue, and the seven largest sellers visible together bring in about 10%. The marketplace has 3,095 sellers.

Decision it supports:
Stock, marketing and supplier risk.

Recommendation:
Protect supply in the leading categories and test growth in the next ranked ones. The marketplace does not rely on any one seller, which lowers supplier risk.

Insight 8: Customers rarely come back.

Evidence:
The Task 2 profiling shows 99,441 orders placed by 96,096 different customers, about 1.03 orders per customer.

Decision it supports:
Whether to invest in new or returning customers.

Recommendation:
Start a repeat purchase programme with an offer after the first delivery, and measure the repeat rate each quarter.

8.2 How the Insights Support Decisions

Area	Insights	Decision supported
Sales and marketing	1, 2, 3, 5, 8	Where and when to spend, and how to keep customers
Operations and logistics	1, 2, 6	Carrier capacity, peak preparation, delivery promises
Seller and category management	4, 7	Seller recruitment and stock priorities
Regional strategy	3, 4, 5	Where to grow and what to fix first

8.3 Limitations

- The data covers about two years, and the first and last months hold very few orders, so the analysis uses January 2017 to August 2018. The 2018-Q3 figures are a partial quarter.
- There is no cost or profit data, so margins cannot be measured.
- Customers and sellers are anonymised and shown as numbers assigned by the warehouse.
- Review comments were not loaded, so the reasons behind low scores cannot be analysed.
- The links between delivery time and satisfaction are associations in time, not proof of cause. Seasonal conclusions rest on a single November.

Acknowledgement of AI Usage

Generative AI (Claude, by Anthropic) was used as a supporting tool in this assignment, following the CLEAR framework. It was used to explain concepts, suggest the structure of SQL scripts and ETL procedures, help debug errors (for example in SSIS, SSAS deployment and cube processing), give guidance on building the Power BI dashboard, and proofread parts of the report.

All AI output was reviewed, tested and adapted by the group. Every script was run on our own SQL Server, the ETL was checked with automated validation, and all figures in this report come from our own queries, cube and dashboard. Suggestions that were wrong or did not fit our setup were rejected or corrected. The design decisions, implementation and insights reflect the group's own understanding, and the group takes full responsibility for the submission.